

## **Grid Deep Dive : GPU Performance Portability Training**

Peter Boyle, Brookhaven National Laboratory



- What is a GPU really?
- Grid and GPU portability
  - How to program multiple (moving) targets?
  - Performance on Exascale systems
- Use of Batched GEMM in LQCD (multigrid)
- Parallel FFT case study
- **Use & power of coding agents**

[www.github.com/paboyle/Grid](https://www.github.com/paboyle/Grid)



Frontier AMD CPU, AMD GPU; [HIP](#)



Perlmutter AMD CPU, Nvidia GPU; [CUDA](#)



Aurora Intel CPU, Intel GPU; [SYCL](#)



Summit IBM CPU, Nvidia GPU; [CUDA](#)

+ CPU computing is also *not* going away

- All major US supercomputers are GPU accelerated;
- + many or most recent EU installations: Lumi-G, Booster, Jupiter, PizDaint, Tursa etc..
- There are multiple, **software incompatible platforms**
- Code must be highly parallel, regular (lots work items executing same software paths)
- Data must be structured and laid out to create locality of access between work items
  - Used to be called vectorization, but the software interfaces complicate this a little.
- Native GPU APIs are C++ family
- Huge factors of computer performance will be lost if algorithms/software do not face these facts.
- Not all algorithms will naturally suit GPUs. Life is tough.

If the internal abstraction is defined right, porting is easy !

- Performed the HIP and SYCL ports of Grid on *the SAME DAY*

What is a GPU?

# SIMD vs SIMT is the main programming model difference

The GPU premise is to provide hardware for executing 3D (6D) loops

Each loop iteration appears as a distinct 'software thread'

Each thread is told which 3D iteration tuple it must work on.

Loop iterations that are 'close by' other can communicate with each other and MUST coordinate memory accesses for efficiency.

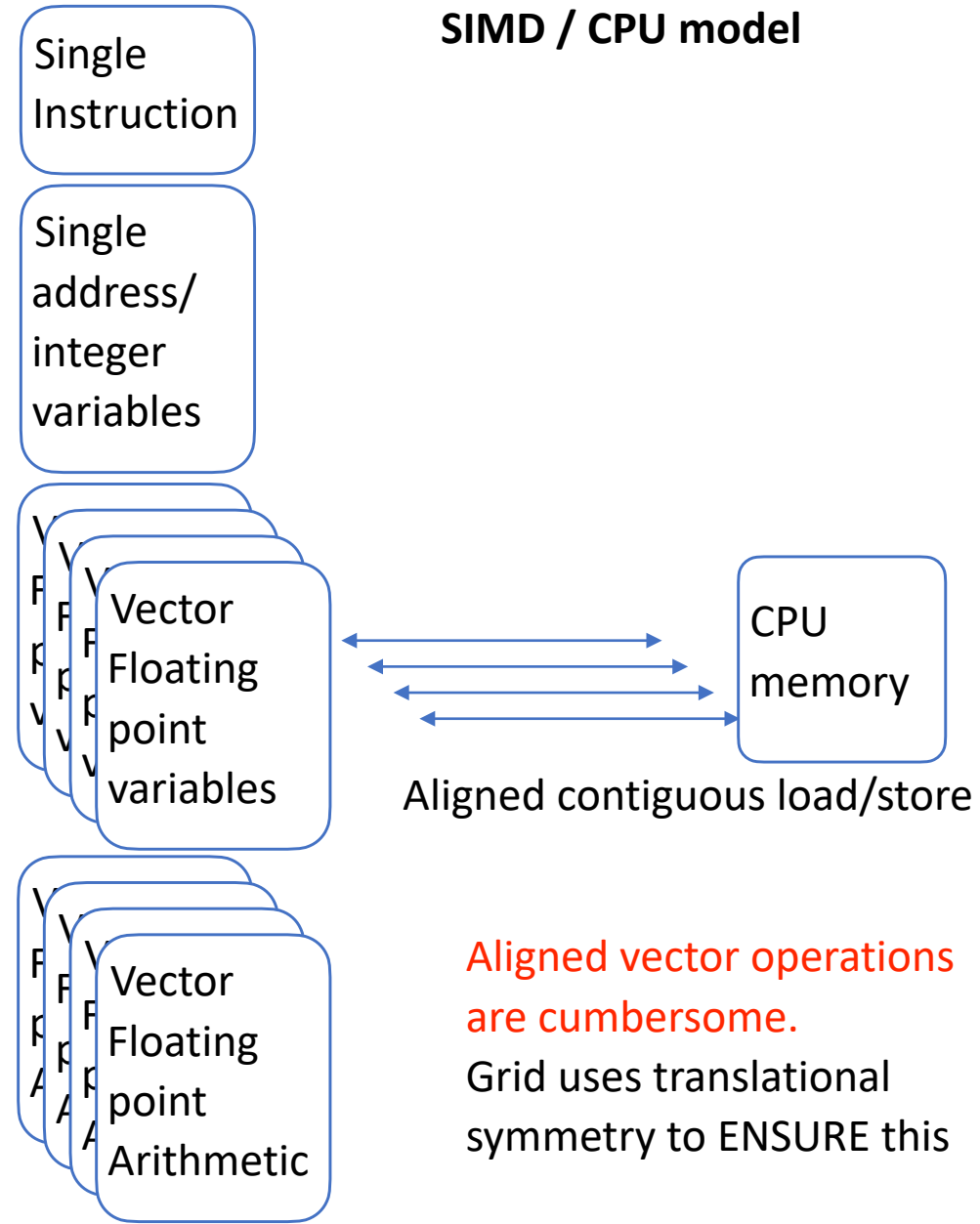
**SIMD = Single Instruction Multiple Data = CPU vectorization**

**SIMT = Single Instruction Multiple Thread = GPU hidden vectorization**

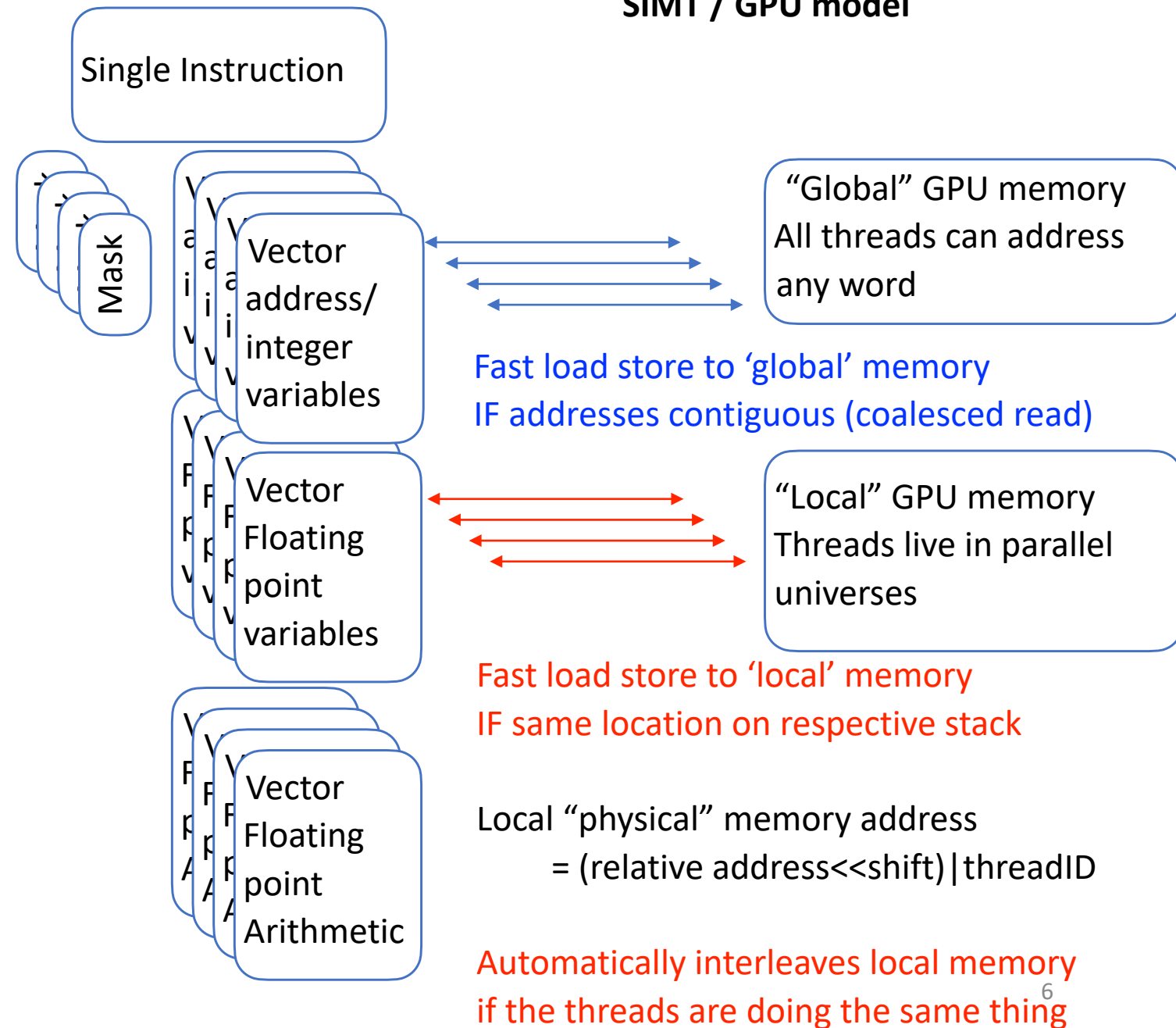
SIMT in many ways makes vectorization easier and more automatic

but you need to get your head around what is happening and penetrate some jargon

## SIMD / CPU model



## SIMT / GPU model



# Covariant programming : capturing the variation between SIMD and SIMT in a single code

The struct-of-array (SoA) portability problem:

- Scalar code: CPU needs struct memory accesses struct calculation
- SIMD vectorisation: CPU needs SoA memory accesses and SoA calculation
- SIMT coalesced reading: GPU needs SoA memory accesses struct calculation
- GPU data structures in memory and data structures in thread local calculations *differ*

| Model   | Memory                       | Thread                   |
|---------|------------------------------|--------------------------|
| Scalar  | Complex Spinor[4][3]         | Complex Spinor[4][3]     |
| SIMD    | Complex Spinor[4][3][N]      | Complex Spinor[4][3][N]  |
| SIMT    | Complex Spinor[4][3][N]      | Complex Spinor[4][3]     |
| Hybrid? | Complex Spinor[4][3][Nm][Nt] | Complex Spinor[4][3][Nt] |

**How to program portably?**

- Use operator() to transform memory layout to per-thread layout.
- Two ways to access for read
- operator[] returns whole vector
  - operator() returns SIMD lane threadIdx.y in GPU code
  - operator() is a trivial identity map in CPU code
- Use coalescedWrite to insert thread data in lane threadIdx.y of memory layout.

# Wilson Kernels

```
#define GENERIC_STENCIL_LEG(Dir,spProj,Recon)
SE = st.GetEntry(ptype, Dir, sF);
if (SE->_is_local) {
    int perm= SE->_permute;
    auto tmp = coalescedReadPermute(in[SE->_offset],ptype,perm,lane);
    spProj(chi,tmp);
} else {
    chi = coalescedRead(buf[SE->_offset],lane);
}
acceleratorSynchronise();
Impl::multLink(Uchi, U[sU], chi, Dir, SE, st);
Recon(result, Uchi);
```

Recovers parallel execution after thread divergence in if/else for boundary / interior link.

Multiplying by gauge link is provided by a “policy” template class.

```
template <class Impl> accelerator_inline
void WilsonKernels<Impl>::GenericDhopSiteDag(StencilView &st, DoubledGaugeFieldView &U,
    SiteHalfSpinor *buf, int sF,
    int sU, const FermionFieldView &in, FermionFieldView &out)
{
    typedef decltype(coalescedRead(buf[0])) calcHalfSpinor;
    typedef decltype(coalescedRead(in[0])) calcSpinor;
    calcHalfSpinor chi;
    // calcHalfSpinor *chi_p;
    calcHalfSpinor Uchi;
    calcSpinor result;
    StencilEntry *SE;
    int ptype;
    const int Nsimd = SiteHalfSpinor::Nsimd();
    const int lane=acceleratorSIMTlane(Nsimd);
    GENERIC_STENCIL_LEG(Xp,spProjXp,spReconXp);
    GENERIC_STENCIL_LEG(Yp,spProjYp,accumReconYp);
    GENERIC_STENCIL_LEG(Zp,spProjZp,accumReconZp);
    GENERIC_STENCIL_LEG(Tp,spProjTp,accumReconTp);
    GENERIC_STENCIL_LEG(Xm,spProjXm,accumReconXm);
    GENERIC_STENCIL_LEG(Ym,spProjYm,accumReconYm);
    GENERIC_STENCIL_LEG(Zm,spProjZm,accumReconZm);
    GENERIC_STENCIL_LEG(Tm,spProjTm,accumReconTm);
    coalescedWrite(out[sF],result,lane);
};
```

Share kernel source for BOTH flavored G-parity / C\* and periodic

These datatypes change **covariantly** With the architecture for SIMD vs SMT

Contrast to most portability approaches where software is **invariant**

## Thread Divergence - GPU's HATE branchy code

The thread enable mask is important!

If some threads say “IF” and others say “ELSE” then **both** IF and ELSE are computed with enable masks set accordingly

- Parallel execution rapidly becomes serial execution in branchy code
  - A program can be thought of as a binary tree of branch decisions
    - Highly divergent code will have to execute down to ALL leaves of the b-tree
  - Bad news for a lot of HEP event processing code, GEANT etc...
    - (Added to problem of object orientation and virtual pointers do not work in GPU memory systems)

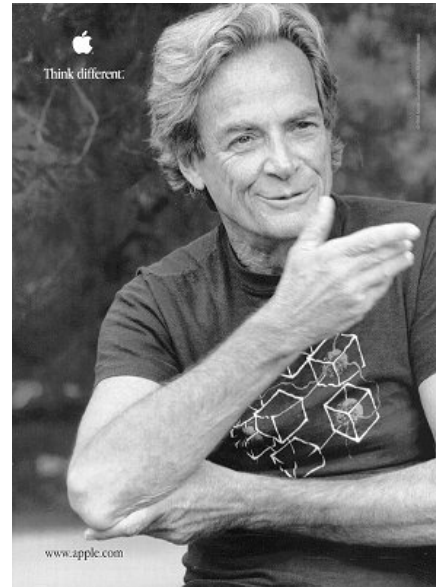
## GPU register files are huge.

- Allows to schedule loads earlier and cover latency
- Many threads sharing same execution pipes increases latency tolerance
  - Reminder / humble-brag : I designed the adaptive prefetch engine on IBM BlueGene/Q - this also covers latency
- Register files are multi-banked (unlike CPU) with access combination rules that only the compiler really handles well

# Grid library for structured calculations

- How to portably target highly parallel hardware ?
  - Easiest if the syntax is high level, and then broken up under an interface
  - Data parallel *used* to mean more than concurrently executed loop bodies – treat entire arrays as objects
  - “Borrow” concepts from the *Connection Machine*

CM-2



<https://longnow.org/ideas/richard-feynman-and-the-connection-machine/>

# High level data parallelism

- At the highest level deal with a distributed array across a multi-node computer as an aggregate object
  - Periodic CSHIFT primitives
  - Conformable array operations on arrays of identically laid out data
    - c.f. F90 syntax array wildcarding/slicing
  - Predicated data parallel operation (where)
- High Performance Fortran was a failure due to MPI latency
  - Introduce additional, lower-level support for PDE stencil operations & halo exchange.
- Use C++ opaque containers for distributed arrays
  - Operator overloading leaves expressions “natural”
  - (compact C++11, 200 LoC) expression template engine keeps site local expressions optimized.
  - HIP/CUDA/SYCL native APIs are all C++ domain ; reduced technical risk if use native API
- Supports CPU w/ SIMD, CUDA, SyCL 2020, HIP compilation targets
  - Device data motion uses either software cache or unified virtual memory

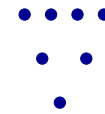
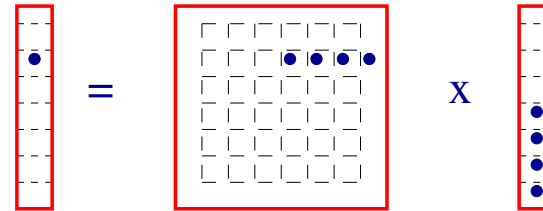
```
A=B*C+Cshift(D,1,Xdir);
```

```
face = (mod(coor,Block[mu]) == 0) ;
```

```
projected= where(face==1,sp_proj,projected);
```

# Unifying CPU and GPU programming models

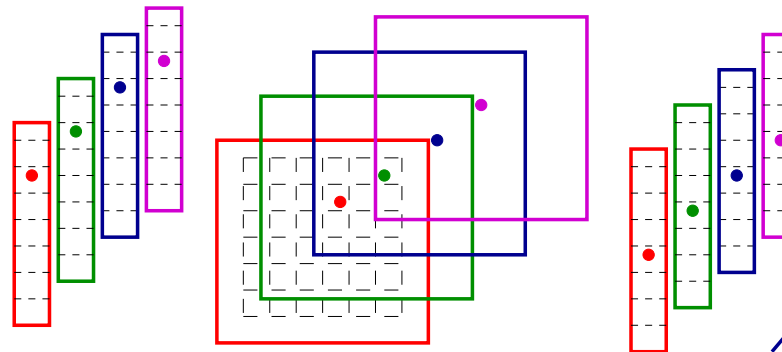
Vector = Matrix x Vector



Reduction of vector sum is bottleneck for small N

- Memory layout:
  - **both** SIMD and SIMT need a vector direction
- Optimised stencil code will see lower-level interfaces
- SIMD and SIMT becomes exposed

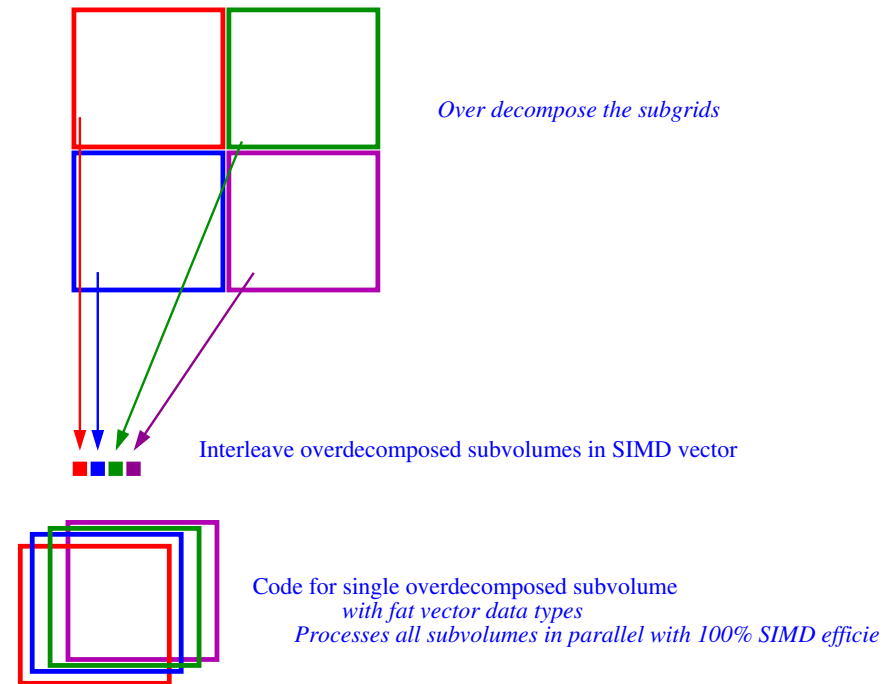
Many vectors = many matrices x many vectors



No reduction or SIMD lane crossing operations.

SIMD interleave

- Geometrically decompose cartesian arrays across nodes (MPI)
- Subdivide node volume into smaller *virtual nodes*
- Spread virtual nodes across SIMD lanes
- Use OpenMP+MPI+SIMD to process conformable array operations
- Same instructions executed on many nodes, each node operates on four virtual nodes



- Conclusion: Modify data layout to align data parallel operations to SIMD hardware
- Conformable array operations simple and vectorise **perfectly**

# Portability 101: Use an internal abstraction

- Grid contains a **data parallel API**, **expression template engine**, general QFT data types
- **D-dimensional Lattice<T> containers** have opaque layout and partial vector layout transformation
  - Expression template engine + Circular shift (Cshift) allows high level code
- **Internal offload API** in ``Accelerator.h``
  - `accelerator_for()` macro **captures a loop body in a C++11 Lambda function** and executes it.
  - `acceleratorCopyToDevice` etc....
  - `deviceVector<T>`
- **Software managed cache** of Lattice objects is maintained **on device**.
  - Lookup in **cache is O(1) overhead** (hash table - cost does NOT grow with volume of data)
  - **True LRU Q** (least recently used) eviction algorithm when a high-watermark is exceeded
  - Transient / discard next facility
  - Entries are “CpuDirty”, “AccDirty”, “Consistent” ; “Views” can be open for CPU or GPU but not both
    - Opening views updates state and triggers data motion
- API covers **OpenMP, HIP, SYCL and CUDA**: vector intrinsics for most CPUs and most major GPUs.
  - **Most users do not see this API** because the Lattice ET engine sits above it.
    - API has also been used directly by C. Lehner (GPT), C. Kelly (CPS), Luchang Jin (Qlat)
    - It's **open source and can be mined for ideas**

```

#define accelerator_for2dNB( iter1, num1, iter2, num2, nsimd, ... ) \
{ \
    int nt=acceleratorThreads(); \
    typedef uint64_t Iterator; \
    auto lambda = [=] accelerator \
        (Iterator iter1,Iterator iter2,Iterator lane) mutable { \
        __VA_ARGS__; \
    }; \
    dim3 cu_threads(nsimd,acceleratorThreads(),1); \
    dim3 cu_blocks ((num1+nt-1)/nt,num2,1); \
    LambdaApply<<<cu_blocks,cu_threads,0,computeStream>>>(num1,num2,nsimd,lambda); \
}

template<typename lambda> __global__
void LambdaApply(uint64_t num1, uint64_t num2, uint64_t num3, lambda Lambda)
{
    // Weird permute is to make lane coalesce for large blocks
    uint64_t x = threadIdx.y + blockDim.y*blockIdx.x;
    uint64_t y = threadIdx.z + blockDim.z*blockIdx.y;
    uint64_t z = threadIdx.x;
    if ( (x < num1) && (y<num2) && (z<num3) ) {
        Lambda(x,y,z);
    }
}

```

CUDA/Nvidia

```

accelerator_inline int acceleratorSIMTlane(int Nsimd) {
#ifdef GRID_SIMT
    return __spirv::initLocalInvocationId<3, cl::sycl::id<3>>(){}[2];
#else
    return 0;
#endif
} // SYCL specific

#define accelerator_for2dNB( iter1, num1, iter2, num2, nsimd, ... ) \
theGridAccelerator->submit([&](cl::sycl::handler &cgh) { \
    unsigned long nt=acceleratorThreads(); \
    if(nt < 8)nt=8; \
    unsigned long unum1 = num1; \
    unsigned long unum2 = num2; \
    unsigned long unum1_divisible_by_nt = ((unum1 + nt - 1) / nt) * nt; \
    cl::sycl::range<3> local {nt,1,nsimd}; \
    cl::sycl::range<3> global{unum1_divisible_by_nt,unum2,nsimd}; \
    cgh.parallel_for( \
        cl::sycl::nd_range<3>(global,local), \
        [=] (cl::sycl::nd_item<3> item) /*mutable*/ \
        [[intel::reqd_sub_group_size(16)]] \
        { \
            auto iter1 = item.get_global_id(0); \
            auto iter2 = item.get_global_id(1); \
            auto lane = item.get_global_id(2); \
            { if (iter1 < unum1){ __VA_ARGS__ } }; \
        }); \
});

#define accelerator_barrier(dummy) { theGridAccelerator->wait(); }

inline void *acceleratorAllocShared(size_t bytes){ return malloc_shared(bytes,*theGridAccelerator);};
inline void *acceleratorAllocDevice(size_t bytes){ return malloc_device(bytes,*theGridAccelerator);};
inline void acceleratorFreeShared(void *ptr){free(ptr,*theGridAccelerator);};
inline void acceleratorFreeDevice(void *ptr){free(ptr,*theGridAccelerator);};

inline void acceleratorCopySynchronise(void) { theCopyAccelerator->wait(); }
inline void acceleratorCopyDeviceToDeviceAsynch(void *from,void *to,size_t bytes) { theCopyAccelerator->memcpy(to,from,bytes);}
inline void acceleratorCopyToDevice(void *from,void *to,size_t bytes) { theCopyAccelerator->memcpy(to,from,bytes); theCopyAccelerator->wait();}
inline void acceleratorCopyFromDevice(void *from,void *to,size_t bytes){ theCopyAccelerator->memcpy(to,from,bytes); theCopyAccelerator->wait();}
inline void acceleratorMemSet(void *base,int value,size_t bytes) { theCopyAccelerator->memset(base,value,bytes); theCopyAccelerator->wait();}

```

SYCL/Intel

Device lambda (same as Kokkos): Capture loop body in ... and inject it into offloaded code as \_\_VA\_ARGS\_\_

```

extern hipStream_t copyStream;
extern hipStream_t computeStream;
/*These routines define mapping from thread grid to loop & vector lane indexing */
accelerator_inline int acceleratorSIMTlane(int Nsimd) {
#ifdef GRID_SIMT
    return hipThreadIdx_x;
#else
    return 0;
#endif
} // HIP specific

#define accelerator_for2dNB( iter1, num1, iter2, num2, nsimd, ... ) \
{ \
    typedef uint64_t Iterator; \
    auto lambda = [=] accelerator \
        (Iterator iter1,Iterator iter2,Iterator lane ) mutable { \
        { __VA_ARGS__;} \
    }; \
    int nt=acceleratorThreads(); \
    dim3 hip_threads(nsimd, nt, 1); \
    dim3 hip_blocks ((num1+nt-1)/nt,num2,1); \
    if(hip_threads.x * hip_threads.y * hip_threads.z <= 64){ \
        hipLaunchKernelGGL(LambdaApply64,hip_blocks,hip_threads, \
            0,computeStream, \
            num1,num2,nsimd, lambda); \
    } else { \
        hipLaunchKernelGGL(LambdaApply,hip_blocks,hip_threads, \
            0,computeStream, \
            num1,num2,nsimd, lambda); \
    } \
}

```

HIP/AMD

```

#define accelerator
#define accelerator_inline strong_inline
#define accelerator_for(iterator,num,nsimd, ... ) thread_for(iterator, num, { __VA_ARGS__ });
#define accelerator_forNB(iterator,num,nsimd, ... ) thread_for(iterator, num, { __VA_ARGS__ });
#define accelerator_barrier(dummy)
#define accelerator_for2d(iter1, num1, iter2, num2, nsimd, ... ) thread_for2d(iter1,num1,iter2,num2,{ __VA_ARGS__ });

accelerator_inline int acceleratorSIMTlane(int Nsimd) { return 0; } // CUDA specific

#define thread_for( i, num, ... ) DO_PRAGMA(omp parallel for schedule(static)) for ( uint64_t i=0;i<num;i++) { __VA_ARGS__ } ;
#define thread_for2d( i1, n1,i2,n2, ... ) \
    DO_PRAGMA(omp parallel for collapse(2)) \
    for ( uint64_t i1=0;i1<n1;i1++) { \
        for ( uint64_t i2=0;i2<n2;i2++) { \
            { __VA_ARGS__ } ; \
        } \
    }

```

OpenMP

Low level code can differentiate “on the device” or “on the host” with GRID\_SIMT macro

## O(1) software managed device cache (MemoryManager)

- Cost does not grow with number of regions managed
    - `std::unordered_map` hash table : maps host pointer (key) to cache entry info (device ptr, state, ref count)
      - Fixed cost lookup, insert, remove
    - `std::list` LRU queue: double linked list with head, tail, fixed cost removal of entries
  - “Lattice” objects allocate host storage but are NOT addressable
    - Obtain “view” with Cpu/Acc read/write to access them
    - ViewOpen/Close drives data migration & state transitions
    - Fully inclusive cache (device is strict subset of host regions)
  - Similar approach as Kokkos and other libraries, e.g. SYCL Buffers, but my own cross platform implementation
- ```
////////////////////////////////////  
// Data tables for ViewCache  
////////////////////////////////////  
typedef std::list<uint64_t> LRU_t;  
typedef typename LRU_t::iterator LRUIterator;  
typedef struct {  
    int          LRU_valid;  
    LRUIterator  LRU_entry;  
    uint64_t     CpuPtr;  
    uint64_t     AccPtr;  
    size_t       bytes;  
    uint32_t     transient;  
    uint32_t     state;  
    uint32_t     accLock;  
    uint32_t     cpuLock;  
} AcceleratorViewEntry;
```

```

// Data tables for ViewCache
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
typedef std::list<uint64_t> LRU_t;
typedef typename LRU_t::iterator LRUiterator;
typedef struct {
    int          LRU_valid;
    LRUiterator  LRU_entry;
    uint64_t     CpuPtr;
    uint64_t     AccPtr;
    size_t       bytes;
    uint32_t     transient;
    uint32_t     state;
    uint32_t     accLock;
    uint32_t     cpuLock;
} AcceleratorViewEntry;

typedef std::unordered_map<uint64_t, AcceleratorViewEntry> AccViewTable_t;
typedef typename AccViewTable_t::iterator AccViewTableIterator ;

static AccViewTable_t AccViewTable;
static LRU_t LRU;

```

```

/*
 * State transitions and actions
 */
 * Action      State      StateNext      Flush      Clone
 *
 * AccRead     Empty      Consistent    -          Y
 * AccWrite    Empty      AccDirty     -          Y
 * AccRead     CpuDirty    Consistent    -          Y
 * AccWrite    CpuDirty    AccDirty     -          Y
 * AccRead     Consistent  Consistent    -          -
 * AccWrite    Consistent  AccDirty     -          -
 * AccRead     AccDirty    AccDirty     -          -
 * AccWrite    AccDirty    AccDirty     -          -
 */

```

Finite state machine  
copies data on/off  
devices as required

```

/*
 * Action   State   StateNext           Flush   Clone
 *
 * CpuRead   Empty   CpuDirty           -       -
 * CpuWrite   Empty   CpuDirty           -       -
 * CpuRead   CpuDirty CpuDirty           -       -
 * CpuWrite   CpuDirty CpuDirty           -       -
 * CpuRead   Consistent Consistent       -       -
 * CpuWrite   Consistent CpuDirty           -       -
 * CpuRead   AccDirty  Consistent       Y       -
 * CpuWrite   AccDirty  CpuDirty           Y       -
 */

```

## Grid performance on Exascale computers

**Benchmark\_usqcd** - produces a CSV spreadsheet of results

- Dslash DWF/ Wilson/ Staggered for various local volumes per MPI rank
- Communication bandwidth (intranode and internode)
- Memory bandwidth
- Batched CGEMM/ZGEMM performance (c.f. multiRHS multigrid: coarseToFine, FineToCoarse, mRHS coarse operator)

|                  | A             | B           |
|------------------|---------------|-------------|
| Memory Bandwidth |               |             |
| Bytes            | GB/s per node |             |
|                  | 6291456       | 254.227647  |
|                  | 100663296     | 3500.053754 |
|                  | 509607936     | 10351.75926 |
|                  | 1610612736    | 14353.04629 |

| Communications |           |      |          |
|----------------|-----------|------|----------|
| Packet bytes   | direction | GB/s | per node |
| 4718592        |           | 2    | 207      |
| 4718592        |           | 3    | 208      |
| 4718592        |           | 6    | 190      |
| 4718592        |           | 7    | 204      |
| 15925248       |           | 2    | 308      |
| 15925248       |           | 3    | 288      |
| 15925248       |           | 6    | 296      |
| 15925248       |           | 7    | 301      |
| 37748736       |           | 2    | 332      |
| 37748736       |           | 3    | 339      |
| 37748736       |           | 6    | 333      |
| 37748736       |           | 7    | 336      |

| ZGEMM |    |     |       |               |             |
|-------|----|-----|-------|---------------|-------------|
| M     | N  | K   | BATCH | GF/s per rank |             |
|       | 16 | 8   | 16    | 256           | 5.787642    |
|       | 16 | 16  | 16    | 256           | 275.941053  |
|       | 16 | 32  | 16    | 256           | 256.925207  |
|       | 32 | 8   | 32    | 256           | 652.809961  |
|       | 32 | 16  | 32    | 256           | 180.69161   |
|       | 32 | 32  | 32    | 256           | 2346.463776 |
|       | 64 | 8   | 64    | 256           | 2338.287944 |
|       | 64 | 16  | 64    | 256           | 4660.337778 |
|       | 64 | 32  | 64    | 256           | 6563.214083 |
|       | 16 | 8   | 256   | 256           | 1950.83907  |
|       | 16 | 16  | 256   | 256           | 3647.22087  |
|       | 16 | 32  | 256   | 256           | 5604.080501 |
|       | 32 | 8   | 256   | 256           | 2874.041285 |
|       | 32 | 16  | 256   | 256           | 5651.272758 |
|       | 32 | 32  | 256   | 256           | 9905.36738  |
|       | 64 | 8   | 256   | 256           | 4444.295629 |
|       | 64 | 16  | 256   | 256           | 8589.934592 |
|       | 64 | 32  | 256   | 256           | 14851.20089 |
|       | 8  | 256 | 16    | 256           | 1928.415632 |
|       | 16 | 256 | 16    | 256           | 3717.942604 |
|       | 32 | 256 | 16    | 256           | 4652.260936 |
|       | 8  | 256 | 32    | 256           | 2995.931429 |
|       | 16 | 256 | 32    | 256           | 5122.814046 |
|       | 32 | 256 | 32    | 256           | 8310.695232 |
|       | 8  | 256 | 64    | 256           | 4350.655689 |
|       | 16 | 256 | 64    | 256           | 8146.75132  |
|       | 32 | 256 | 64    | 256           | 12874.60221 |

# Nvidia Hopper 8x H100 @ SDCC

[illegible]

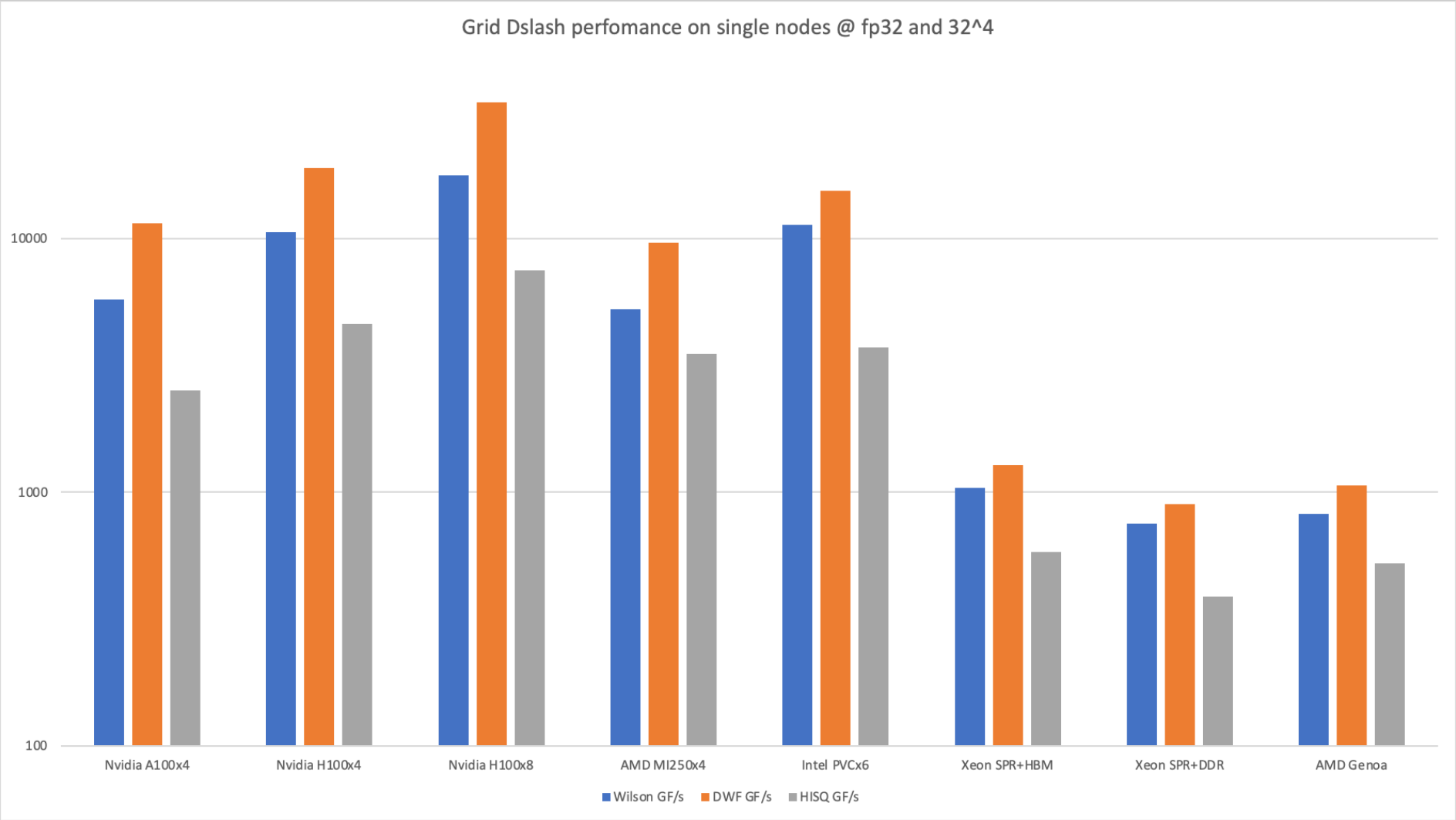
Multi-GPU nodes give between 11TF/s and 40TF/s on AMD, Nvidia and Intel GPU nodes

**Intel PVC:** Aurora@ANL

**AMD MI250:** Frontier@ORNL, Lumi-G@CSC

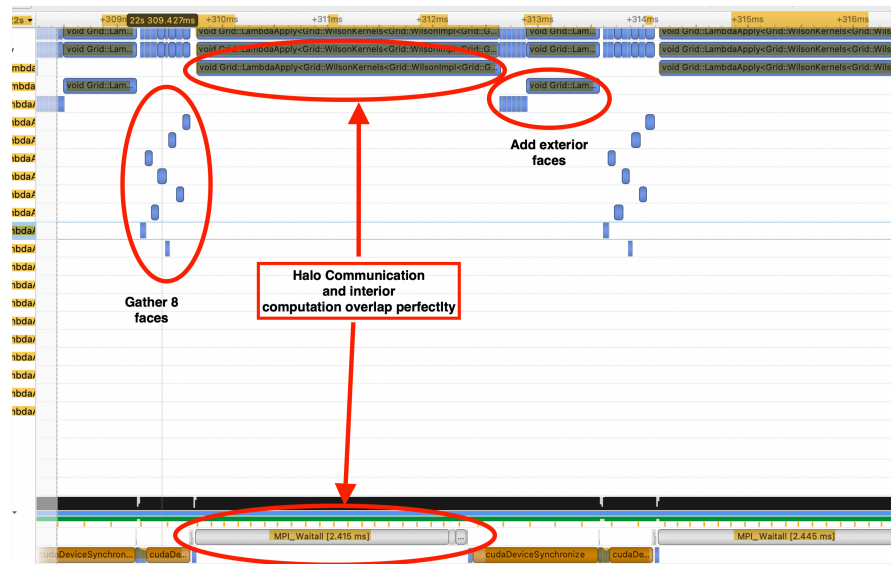
**A100:** Perlmutter@NERSC, Booster@Juelich, Tursa@Edinburgh, Leonard@Cineca

**H100, AMD Genoa:** SDCC@BNL

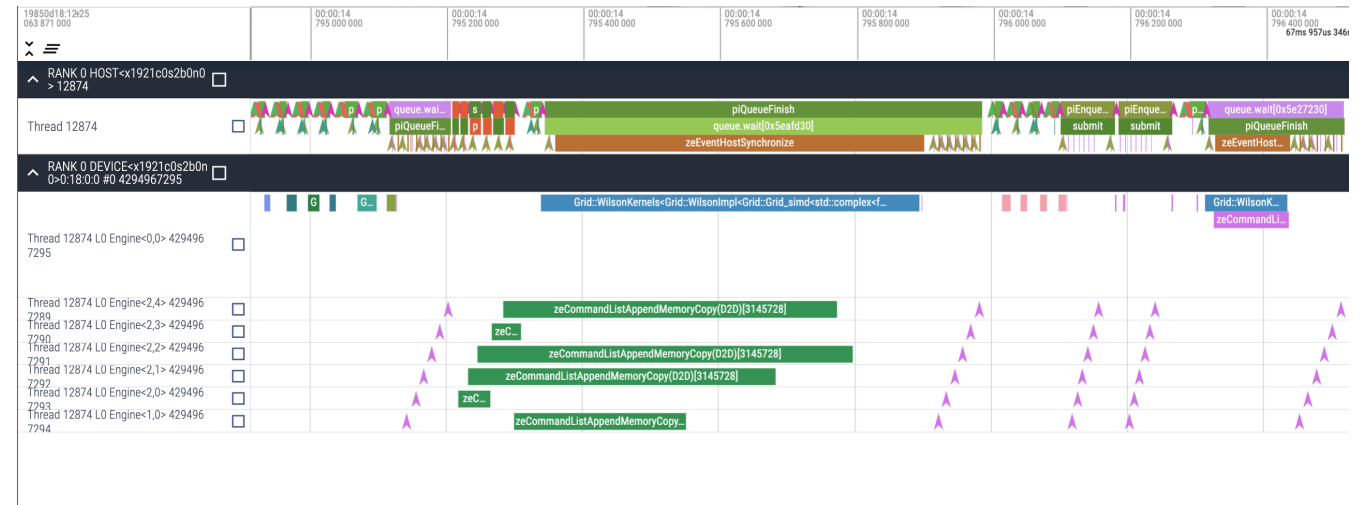


## Communications performance is key to scalability

- **Overlap communication and computation efficiently** from  $16^4$  -  $24^4$  per GPU with 200Gbit/s HFI per GPU
- 180GB/s bidirectional HaloExchange on 4x HPE Slingshot-11-200 OR 4x HDR-200
  - Booster, Tursa, Perlmutter, Frontier all scale very well
- Aurora scales very well with up to 270GB/s bidirectional BW when placed in CPU's HBM (6 HFI's Slingshot-11)
- **Systems that under provision network rapidly hit scaling problems**

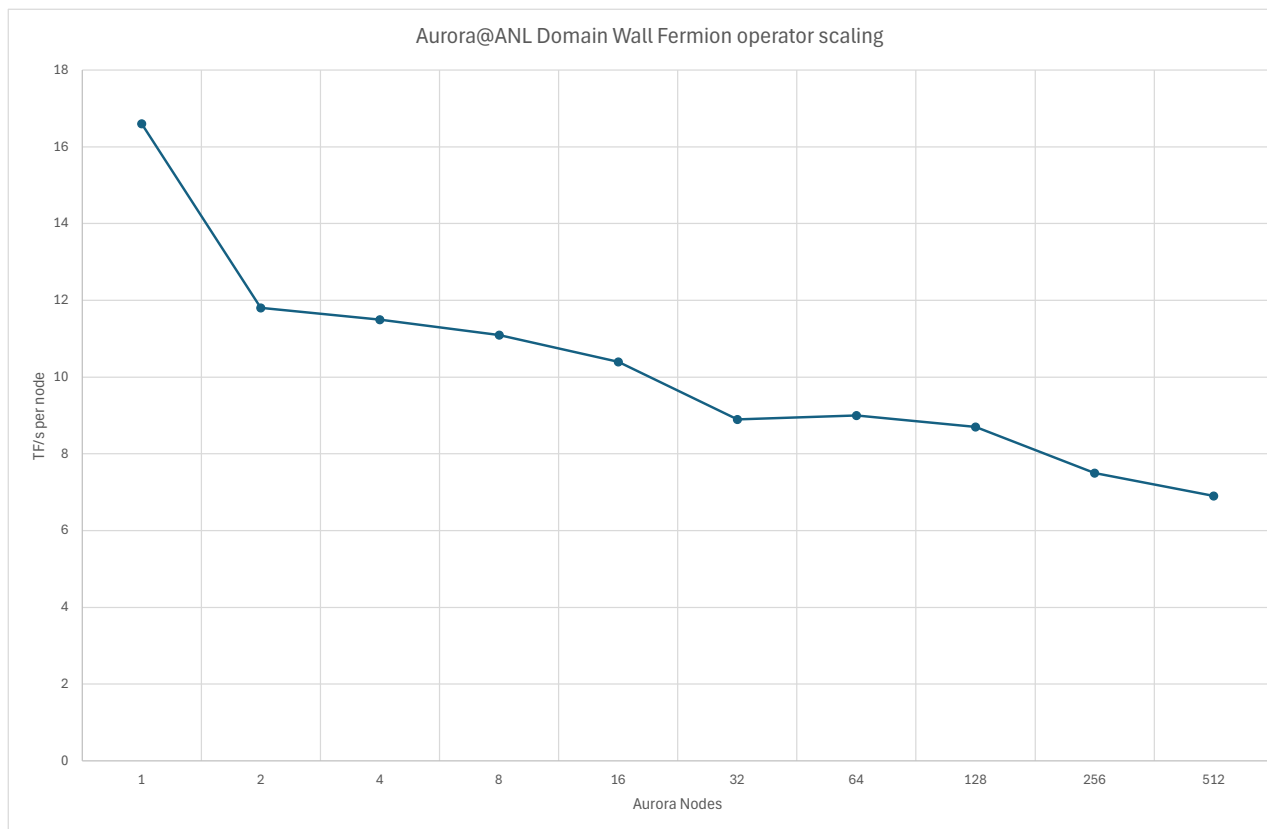
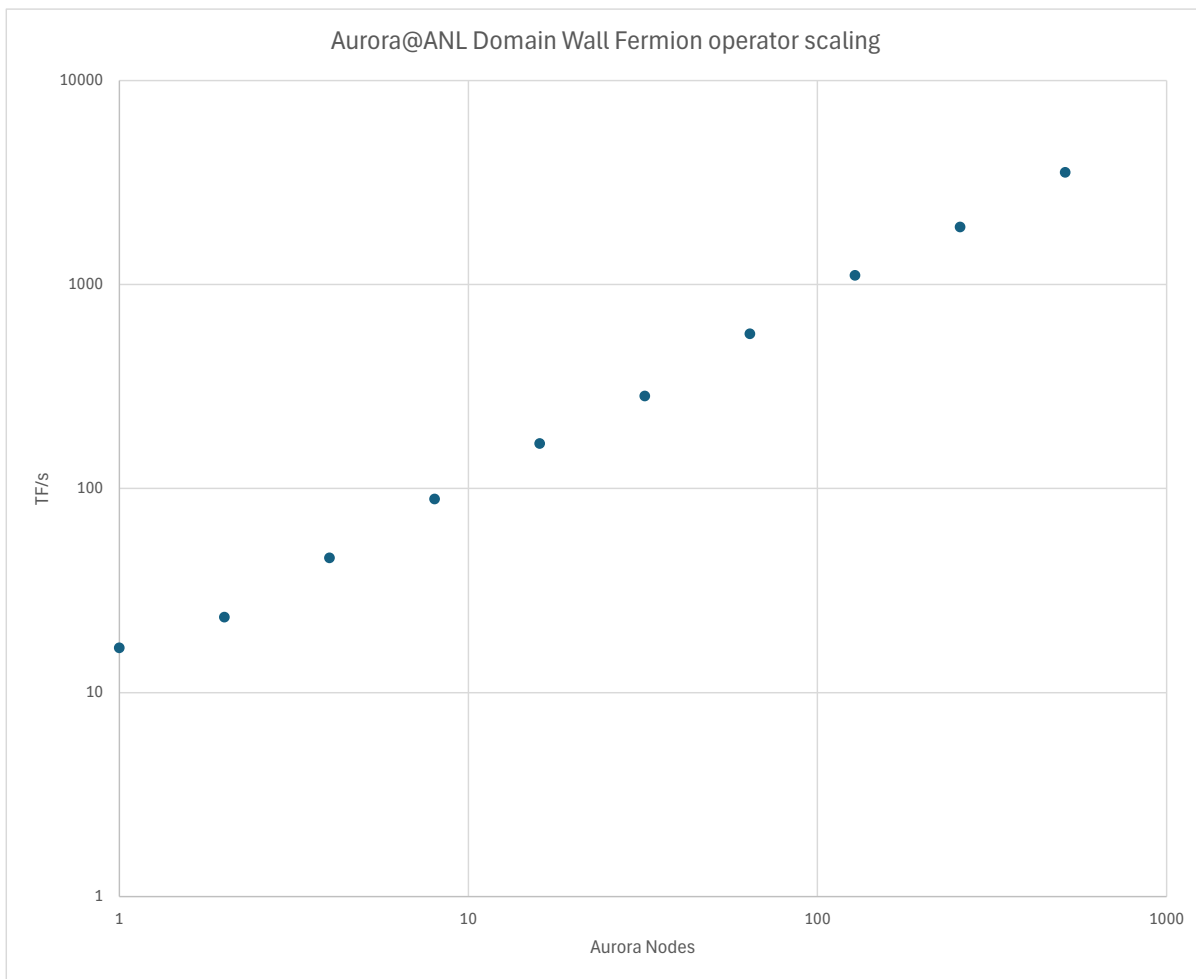


Tursa & Booster  $32^4$  per GPU  
A100 x 4 + HDR-200 x 4  
11TF/s per node fp32



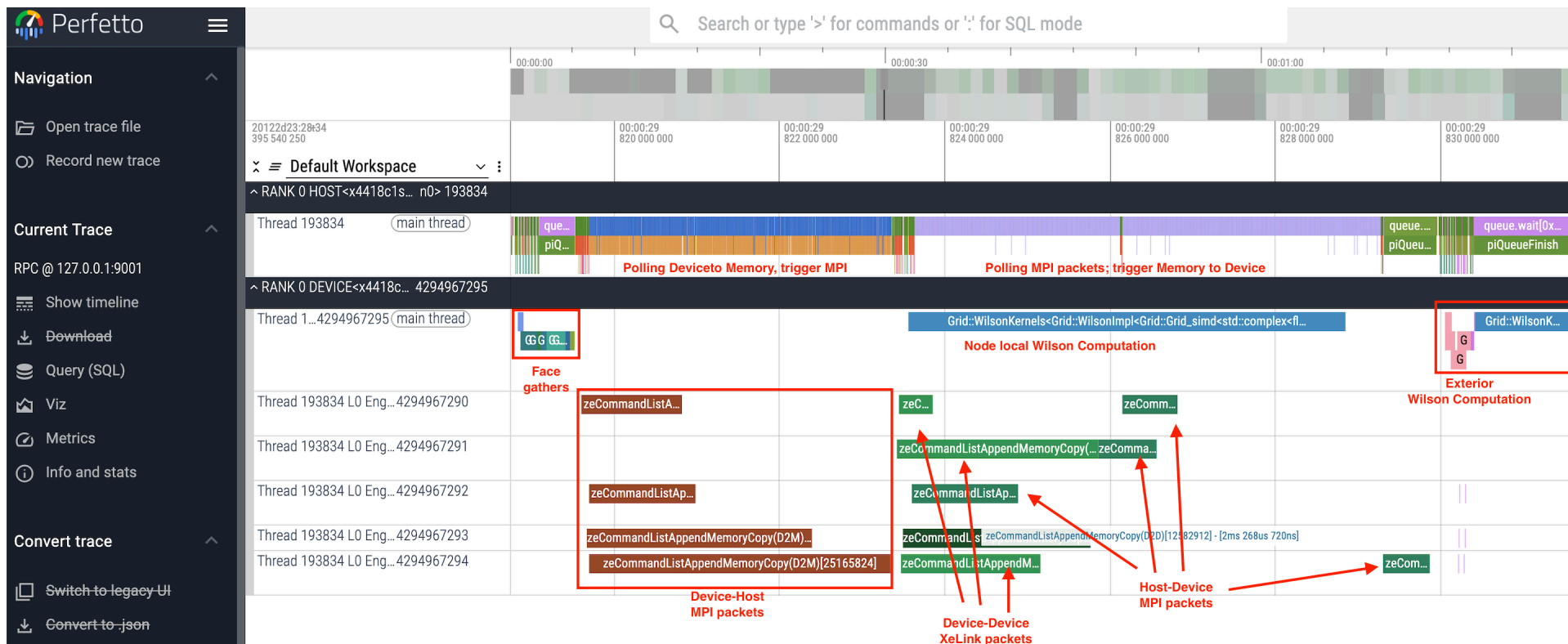
Aurora,  $16^3 \times 32$  per tile  
PVC x 4 + Slingshot x 8  
16-22 TF/s per node fp32 (preliminary)

# Weak scaling on Aurora



Significant effort to reduce dependence on MPI functioning with GPU memory due to bugs on various systems

- Introduced work-around option to always copy to/from host memory before/after MPI
  - <https://github.com/ofiwg/libfabric/issues/11451>
  - <https://github.com/pmodels/mpich/issues/7302>
  - <https://github.com/ofiwg/libfabric/issues/12775>
  - <https://github.com/ofiwg/libfabric/issues/12773>
- Performance enhanced by Grid pipeline mode on Aurora (below)
  - 5D Wilson dslash profile showing significant concurrency and adaptive ordering



## Domain Wall Multigrid (SciDAC-5)

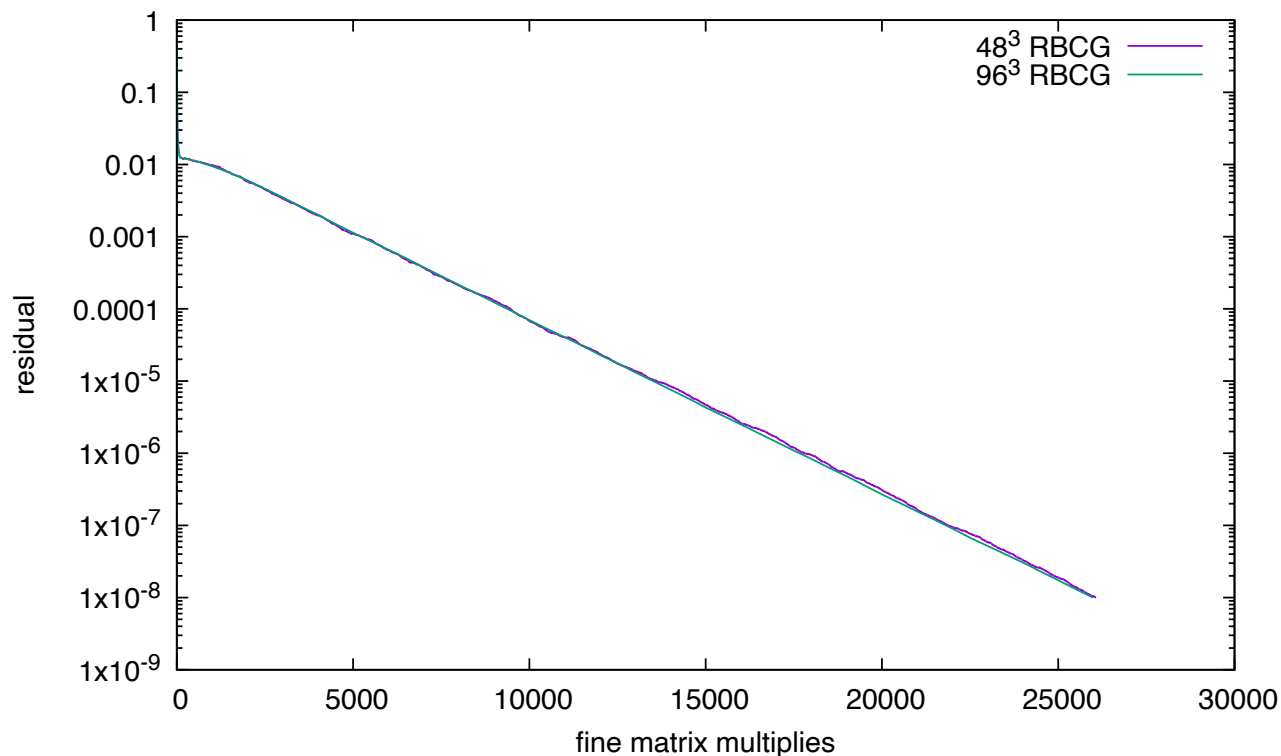
**MDWF Multigrid:** principally discuss: arXiv:2409.03904 and later work

- Multiple right hand side multigrid for domain wall fermions
  - Uses GPU matrix hardware efficiently
  - Introduces multigrid preconditioned block CG for increased algorithm efficiency
  - Over 20x speed up on test system at physical quark masses c.f. red-black CGNE
- CG was this morning's tutorial
  - Use an adaptive multigrid version of this
- Promote matrix vector  $\rightarrow$  matrix matrix via multiple RHS
  - batched GEMM
  - vendor tensor hardware
  - 30% of the run now lives in high performance BLAS across GPU vendors.

## CG (and other Krylov solvers) are a **function only of the spectrum**:

Changing volume 16x didn't affect convergence.

But deflating on 16x bigger volume is a 256 times more expensive (!)



Measured convergence rate in tail:  $\sigma_{cg} = .999451$

Chebyshev bound:  $\sigma = \frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} = 0.999445.$

$$\lambda_{\max} = 89.757 \quad \lambda_{\min} = 6.92238 \times 10^{-6}$$

We are in large volume limit with dense spectrum

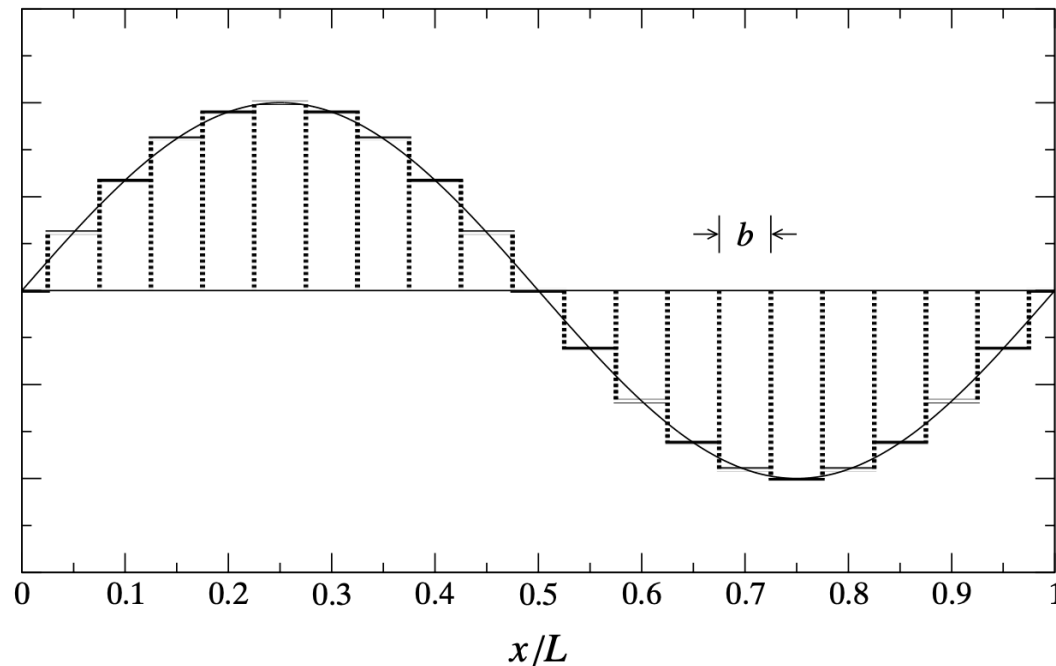
Orders of magnitude more EV's than order of polynomial

Saturates worst case minimax bound

# Multigrid algorithms

## Multigrid Dirac solvers:

- Learn near null space of Dirac matrix ( $\sim 60$  vectors): non-trivial in gauge theory
  - Break vectors into **local wavelet basis** chunks
  - Calculate a representation of Dirac operator within this critical subspace
  - Use as an approximate near-null space multigrid preconditioner
- 
- For DWF fine operator is  $M_{pc}^\dagger M_{pc}$  ; where  $M_{pc} = (M_{ee} - M_{eo}M_{oo}^{-1}M_{oe})$ 
    - 81 point stencil !



(Figure ``borrowed'' from Martin Lüscher's paper)

CERN-PH-TH/2007-096

Local coherence and deflation of the low  
quark modes in lattice QCD

Martin Lüscher

CERN, Physics Department, TH Division  
CH-1211 Geneva 23, Switzerland

## 2 level ADEF-2 preconditioned CG algorithm

### Flexible ADEF2 Preconditioned Conjugate Gradient

1.  $x$  arbitrary
2.  $M^{-1} := (P_R M_{IRS}(\mathcal{H}, \Lambda) + Q)$
3.  $x_0 = Qb + P_R x$
4.  $r_0 = b - \mathcal{H}x_0$
5.  $z_0 = M^{-1}r_0$ ;  $p_0 = z_0$
6. for iteration  $k \in 0, 1, 2 \dots$
7.  $d_k = \mathcal{H}p_k$
8.  $w_k = d_k - \sum_{i=k-m_{max}}^{k-1} \frac{(d_k, \mathcal{H}w_i)}{(w_i, \mathcal{H}w_i)} w_i$
9.  $\alpha_k = (r_k, y_k) / (p_k, w_k)$
10.  $x_{k+1} = x_k + \alpha_k p_k$
11.  $r_{k+1} = r_k - \alpha_k w_k$
12.  $z_{k+1} = M^{-1}r_{k+1}$
13.  $\beta_k = (r_{k+1}, z_{k+1}) / (r_k, z_k)$
14.  $p_{k+1} = z_{k+1} + \beta_k p_k$
15. end for
16. return  $x_{k+1}$

Multigrid cycle is introduced as a CG preconditioner

Flexible algorithm: orthogonalise to previous search directions to tolerate preconditioner variability - if  $A$  is not constant, maintaining  $A$ -orthogonality is hard!

Coarse grid correction

Deflate this with  $O(200)$  coarse grid eigenvectors

Compute using block Lanczos

Smoother - use CG but force it to look at high end of spectrum

$$7 \text{ iterations of CG} \sim \frac{1}{\mathcal{H} + \lambda}$$

Figure 1: Flexible ADEF2 preconditioned conjugate gradient algorithm following Tang et al[41] for solving a Hermitian system  $\mathcal{H}x = b$ , with an additional flexible search direction orthogonalisation (step 8). We have implemented “inexact preconditioned CG” [43] and “flexible CG” [44] variants of the ADEF2 algorithm to address the variability in the preconditioner when it is composed of Krylov processes. The matrix  $Q$  is a coarse grid correction, and the matrix  $M_{IRS}(\mathcal{H}, \Lambda)$  is a smoother in multigrid nomenclature. The Galerkin projector  $P_R$  is defined in the text.

First step: reimplement HDCG in Grid to run on modern GPUs

- Coarse grid was expensive and difficult to optimize on GPUs as not enough work/latency dominated
- Rapidly concluded I needed multiple right hand side implementation

# Why design algorithms for multiple right hand sides?

## Faster code

- Applying the coarse operator is dominated by cost of loading  $n_{\text{basis}}^2$  coefficients  $A_{ij}$  and applying to the  $n_{\text{basis}}$  coarse vector  $\phi_j$
- We can apply to several vectors at once, loading  $n_{\text{basis}}^2$  coefficient matrices and  $n_{\text{basis}} \times n_{\text{rhs}}$ 
  - When  $n_{\text{rhs}} \ll n_{\text{basis}}$  the additional right hand sides are free
  - Promoting to matrix matrix operations increases arithmetic intensity
  - Using multiple right hand sides increases the parallelism in the coarse operations: very GPU friendly

$$\begin{pmatrix} A & & & \\ & A & & \\ & & A & \\ & & & A \end{pmatrix} \begin{pmatrix} \psi_1 \\ \psi_2 \\ \psi_3 \\ \psi_4 \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \end{pmatrix}$$

## Faster algorithms

- Block algorithms, such as BlockCG, can share the Krylov space between multiple solves
  - Known to accelerate staggered fermions; advocated as alternative to multigrid for staggered (Clark et al, 1710.09745)
    - Linear algebra scales as  $n_{\text{rhs}}^2$
    - Matrix multiply scales as  $n_{\text{rhs}}$

### BlockCGrQ

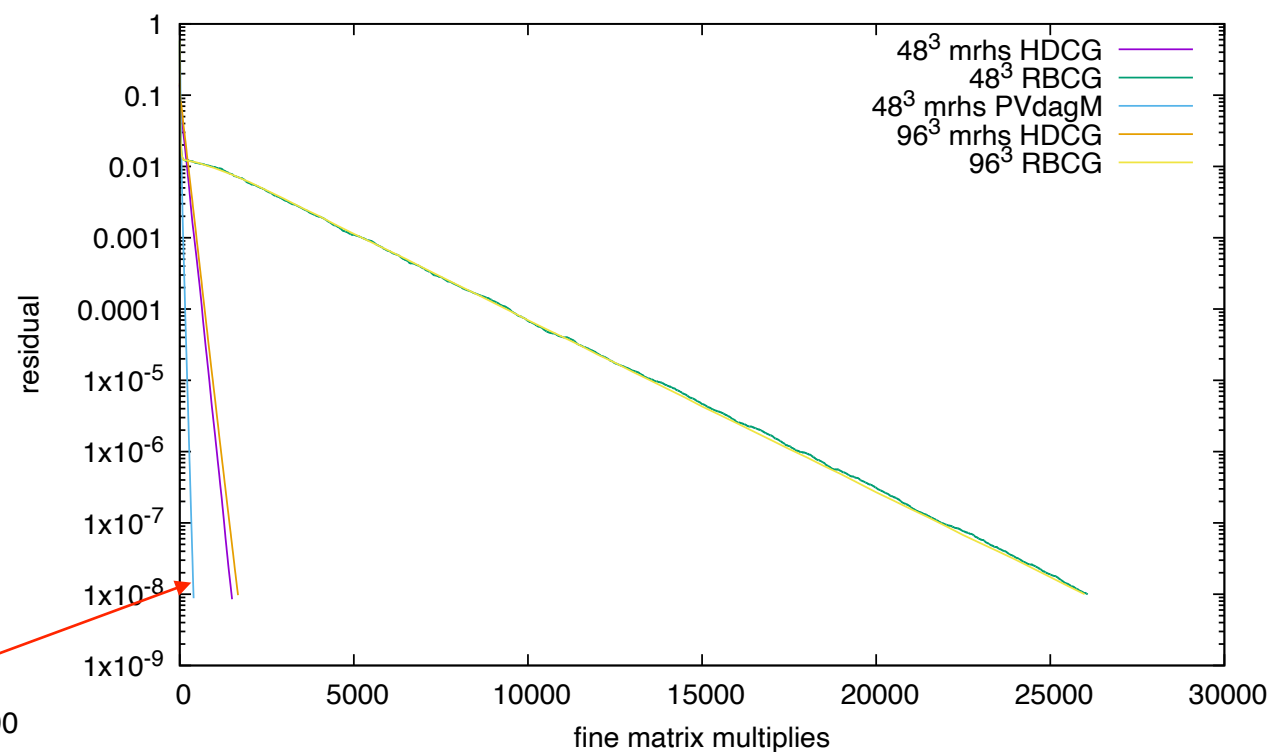
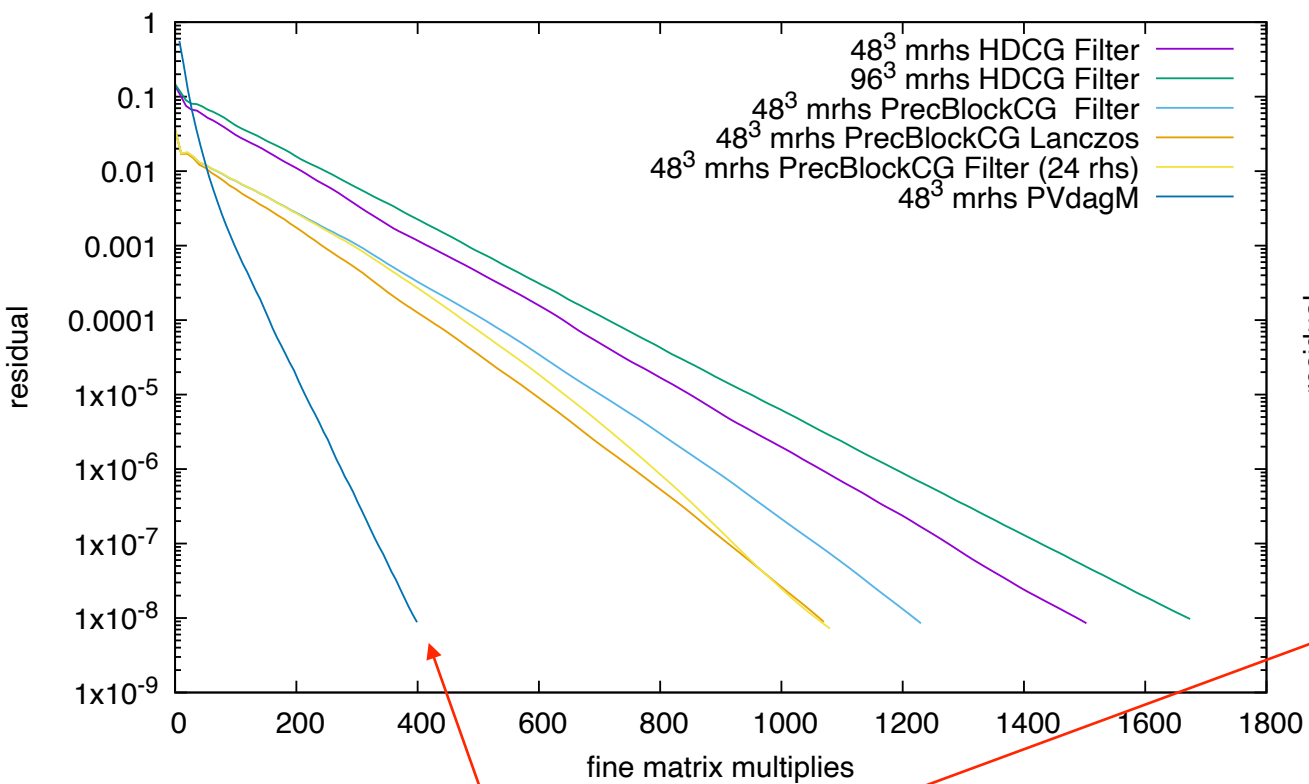
1.  $\mathcal{H} \in \mathbb{C}^{n \times n}$
2.  $B$  source  $\in \mathbb{C}^{n \times n_{\text{rhs}}}$
3.  $X$  arbitrary  $\in \mathbb{C}^{n \times n_{\text{rhs}}}$
4.  $R_0 = B - \mathcal{H}X_0$
5.  $Q_0 C_0 = R_0$
6.  $D_0 = Q_0$
7. for iteration  $k \in 0, 1, 2 \dots$
8.  $Z_k = \mathcal{H}D_k$
9.  $M_k = [D_k^\dagger Z_k]^{-1}$
10.  $X_{k+1} = X_k + D_k M_k$
11.  $Q_{k+1} S_{k+1} = Q_k - Z_k M_k$
12.  $D_{k+1} = Q_{k+1} + D_k S_{k+1}^\dagger$
13.  $C_{k+1} = S_{k+1} C_k$

# Preconditioned Block CG

- To combine multi-grid with block algorithms had to solve two problems
  - Introduce a **preconditioned BlockCGrQ**
    - This **did not exist in the literature**, despite being relatively obvious
  - Address preconditioner variability:  $n_{rhs}^2 \times m_{max}$  orthogonalisation is expensive
    - Replace CG based smoother and coarse solver with a fixed polynomial
- Perform CG in composite matrix  $M^{-1}A$  and with M-inner product
  - $M^{-1}A$  is only Hermitian in the M-inner product  $\langle x|y\rangle_M = \langle x|M|y\rangle_E$
  - The M-inner products are re-written as  $\langle Z|Z\rangle_M = \langle Z|z\rangle_E$

## Preconditioned BlockCGrQ

1.  $z_0 = B - \mathcal{H}X_0$
2.  $Z_0 = M^{-1}z_0$
3.  $C_0^\dagger C_0 = Z_0^\dagger \cdot z_0 = \langle Z_0|Z_0\rangle_M$
4.  $q_0 = z_0 C_0^{-1}$  ;  $Q_0 = Z_0 C_0^{-1}$
5.  $D_0 = Q_0$
6. for iteration  $k \in 0, 1, 2 \dots$
7.  $z_k = \mathcal{H}D_k$
8.  $Z_k = M^{-1}z_k$
9.  $N_k = [D_k^\dagger \cdot z_k]^{-1} = \langle D_k|Z_k\rangle_M^{-1}$
10.  $X_{k+1} = X_k + D_k N_k$
11.  $t_k = q_k - z_k N_k$  ;  $T_k = Q_k - Z_k N_k$
12.  $S_k^\dagger S_{k+1} = T_k^\dagger \cdot t_k = \langle T|T\rangle_M$
13.  $q_{k+1} = t_k S_k^{-1}$  ;  $Q_{k+1} = T_k S_k^{-1}$
14.  $D_{k+1} = Q_{k+1} + D_k S_k^\dagger$
15.  $C_{k+1} = S_k C_k$



New multigrid solver algorithm presented at Lattice 2026:

28x speed up for multi-RHS

14x speed up for single-RHS !

# Significant effort on coarse grid operators in Grid and GPT

- Batched GEMM interface implements 81 point stencil efficiently cross platform
- Communication time is not dominant despite coarse grid (Frontier, 18 nodes)

[Grid](#) / [Grid](#) / [algorithms](#) / blas /

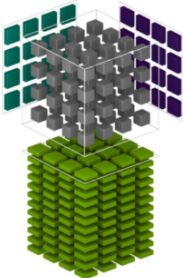
**paboyle** Batched blas, but not working yet on OneAPI

Name

..

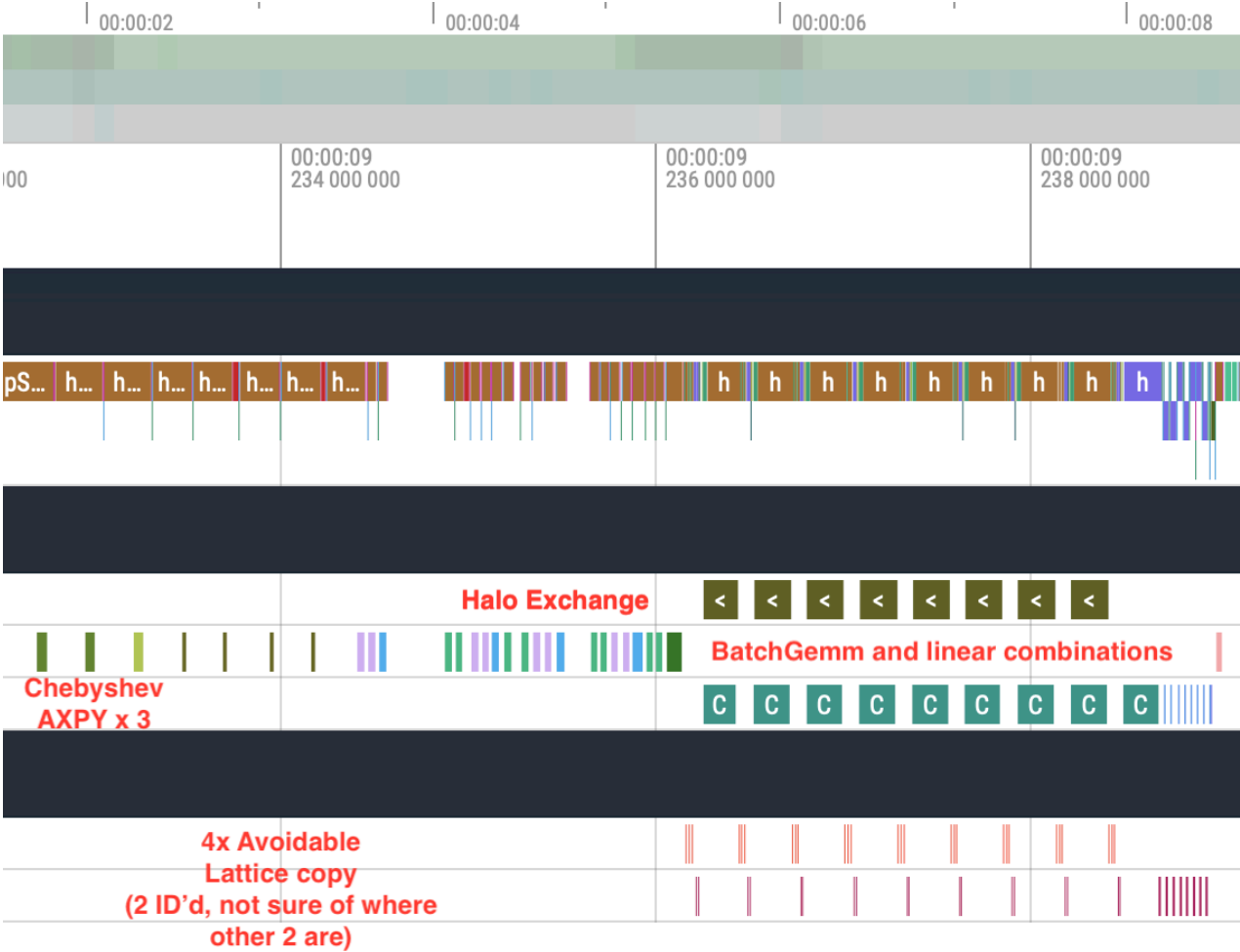
BatchedBlas.cc

BatchedBlas.h

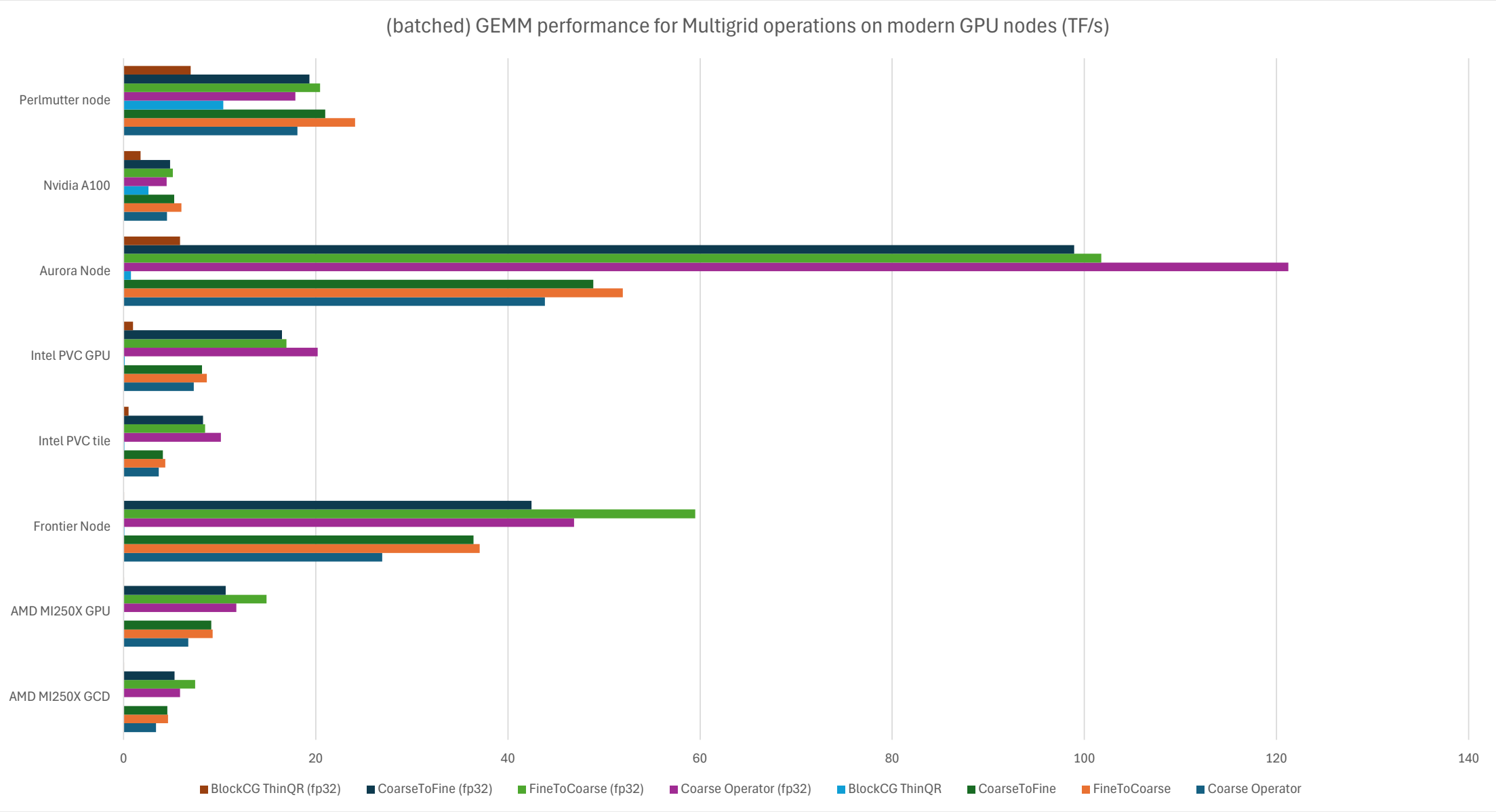


$$D = \begin{bmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \\ A_{31} & A_{32} & A_{33} \end{bmatrix} + \begin{bmatrix} B_{11} & B_{12} & B_{13} \\ B_{21} & B_{22} & B_{23} \\ B_{31} & B_{32} & B_{33} \end{bmatrix}$$

FP16 or FP32      FP16      FP16      FP16 or FP32

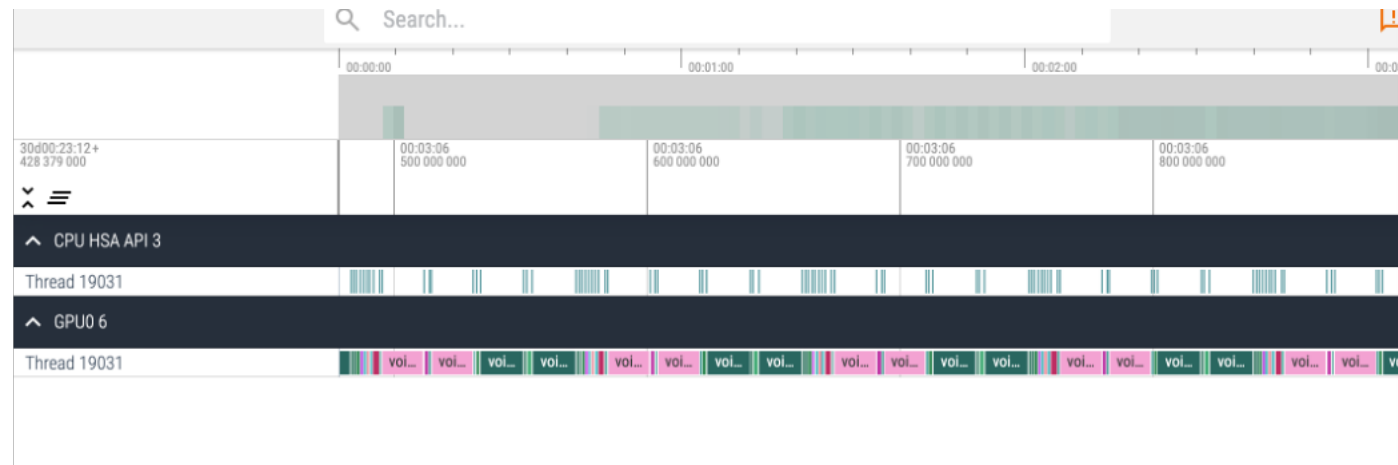


# ZGEMM / CGEMM: Computation part of Coarse grid operators for multiple RHS can perform VERY well



Conjugate Gradient is:

1. Matrix
2. norm
3. linalg
4. Goto 1.



mrhs-Multigrid has

1. 12xCG smoothers
2. BatchedGemm blockProject
3. BatchedGemm deflate
4. MultiRHS coarse CG solve
5. BatchGemm blockPromote
6. Linalg
7. Goto 1.

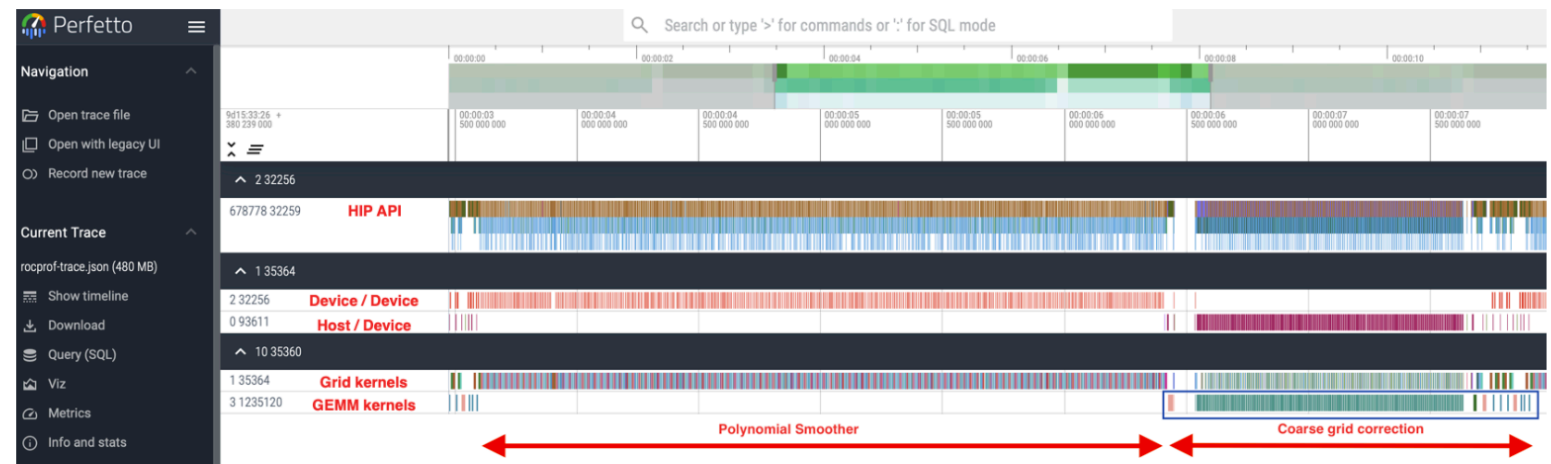
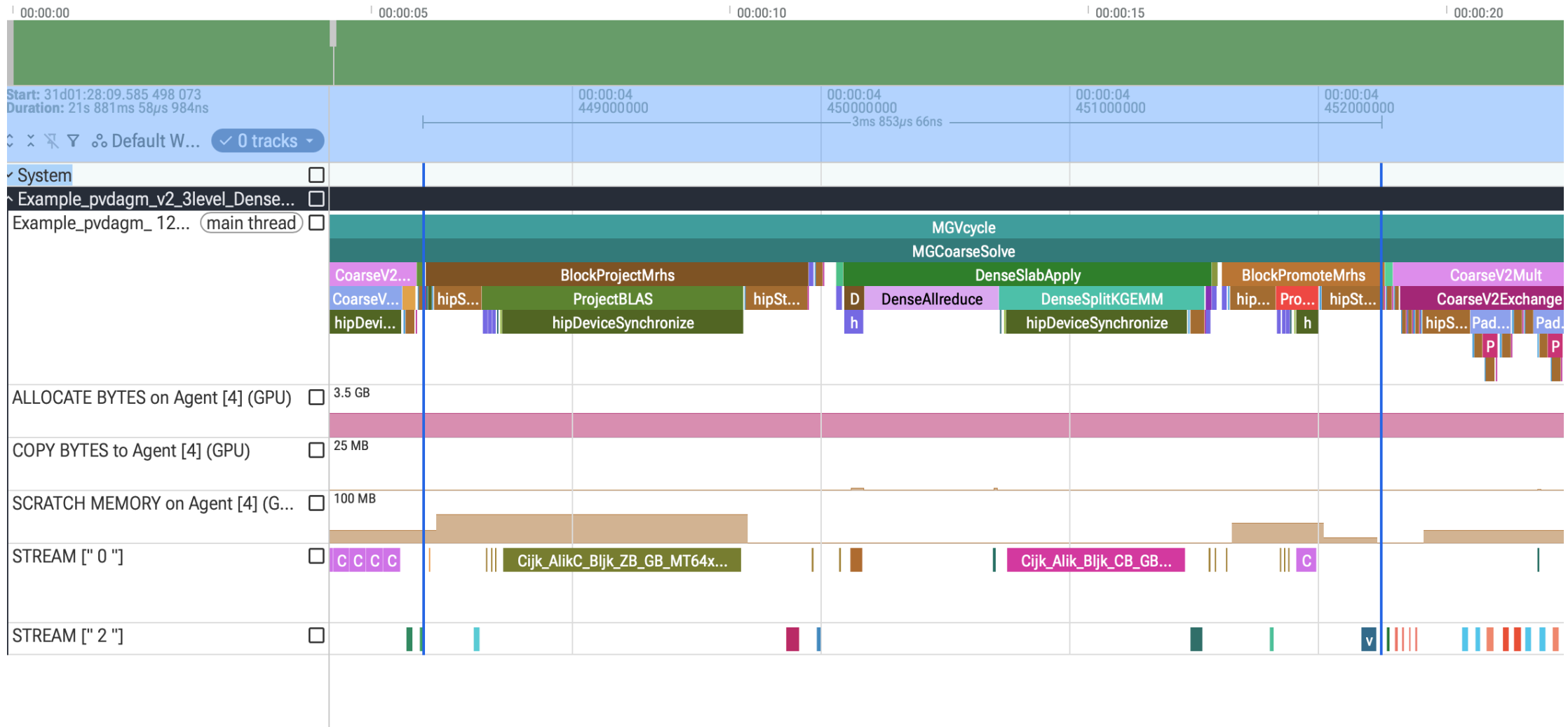


Figure 12: AMD Rocprof obtained profile from Frontier of the multigrid iteration on 18 nodes on the  $48^3$  test problem after careful optimisation. The general Grid software kernels are shown executing along side GEMM kernels, used by projection to coarse, deflation, coarse Chebyshev solver, promotion to fine and then a thinQR rotation. Broadly it is possible to perform almost the entire multigrid preconditioner in BLAS routines using optimised hardware, except for the relatively modest overhead of data layout changes and of course halo-exchange routines.

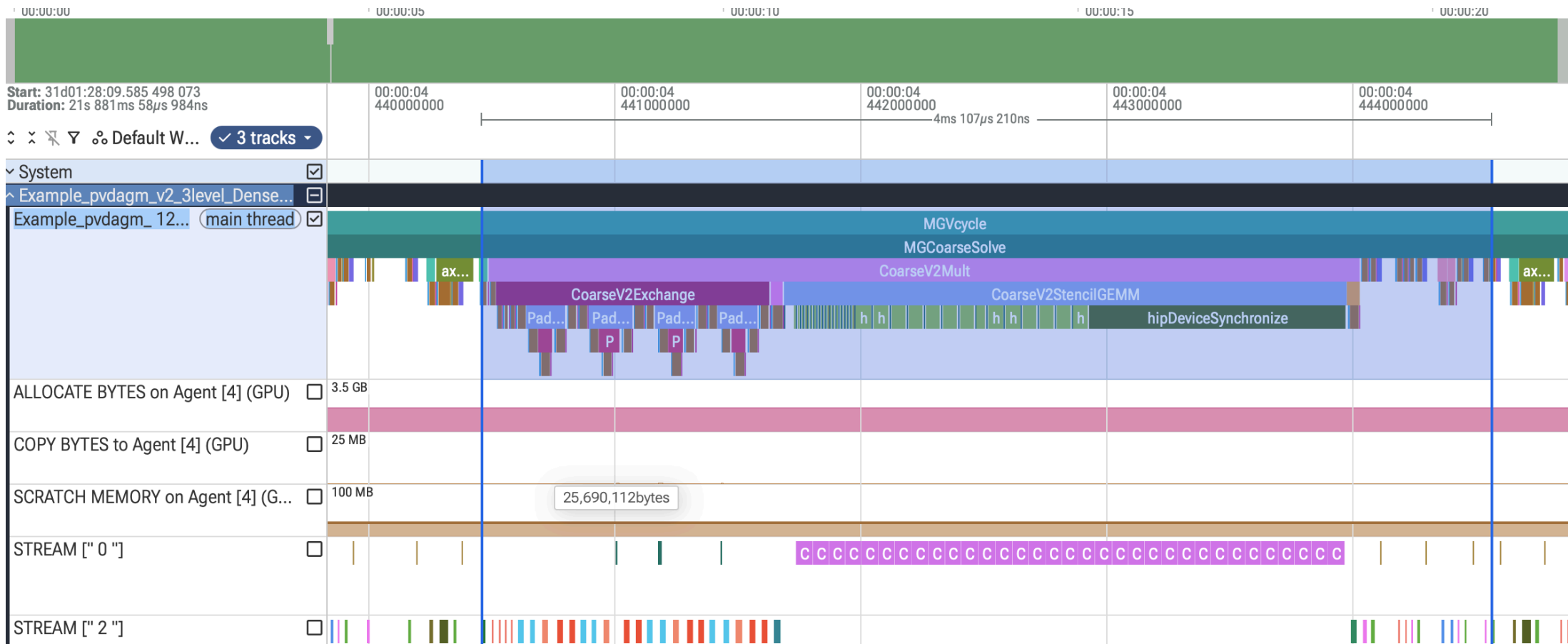
# 3ms exact coarse coarse inverse!

## Uses GPU tensor hardware

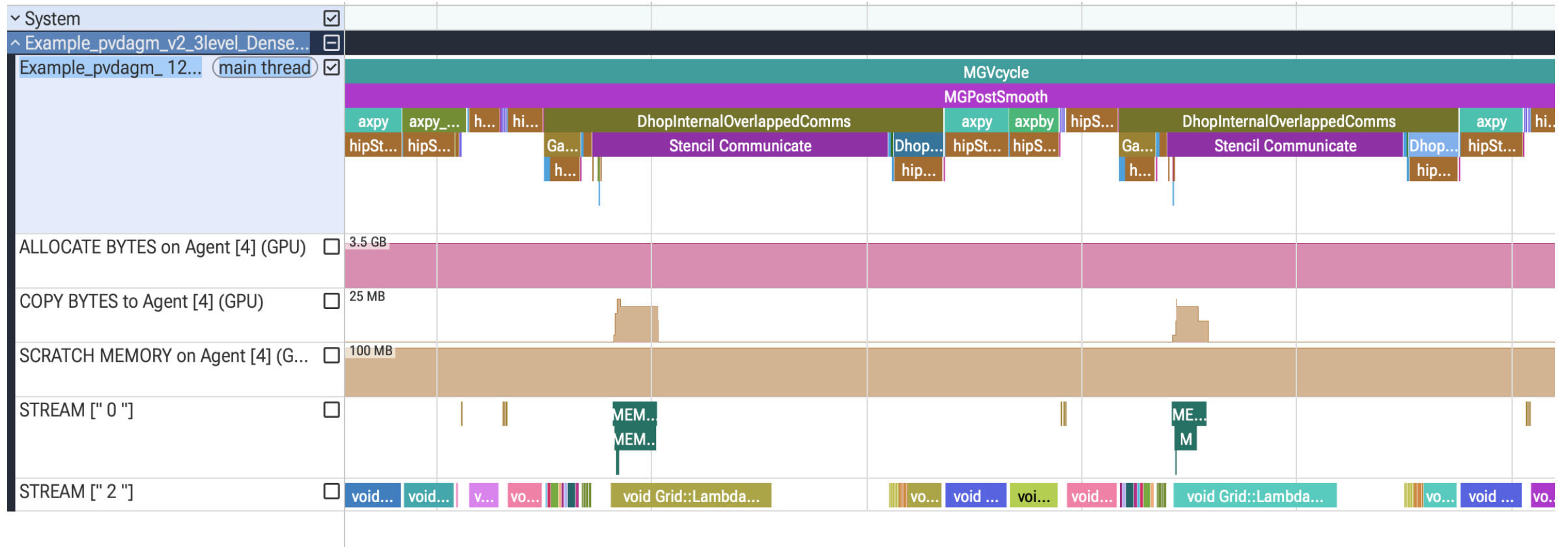


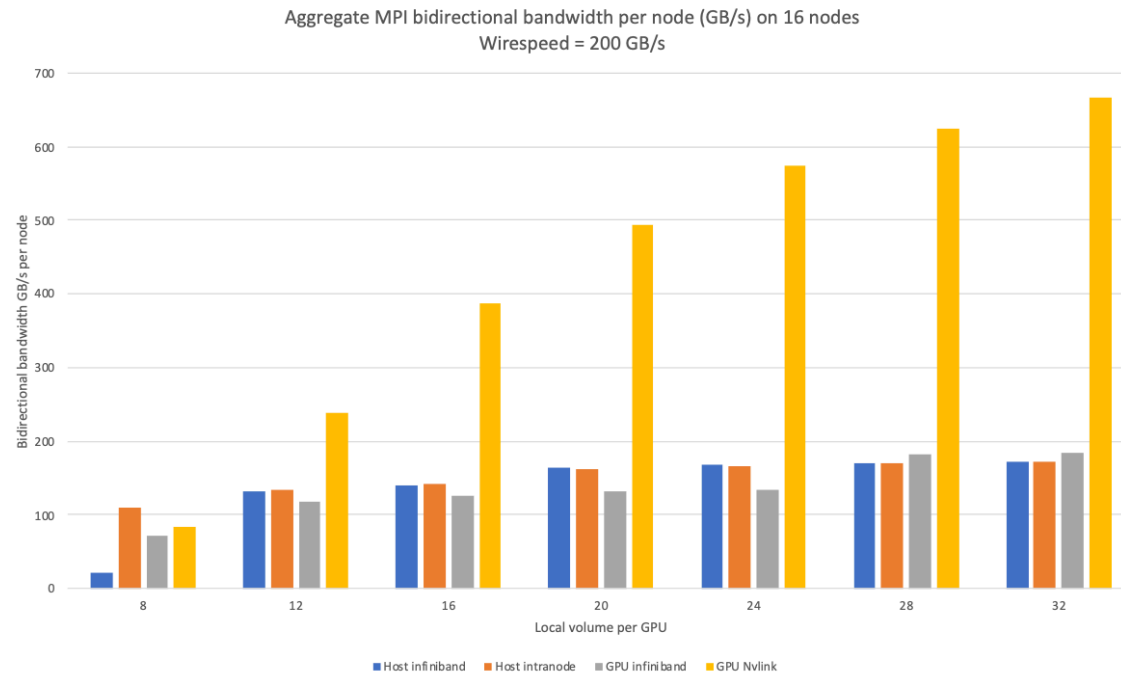
~3ms Coarse operator uses batchGEMM tensor hardware

Communications is efficient despite coarseness & 33 point stencil



# Fine grid Mobius operator was already optimized on Frontier





**Figure 4**

We show the GPU-GPU interconnect performance of four 200Gbit/s Mellanox infiniband cards on Tursa. The peak bidirectional bandwidth is  $4 \times 2 \times 200$  Gbit/s (and so 200 GB/s). Over 90% of this speed is delivered using MPI to access GPU Direct RDMA between GPUs. NVlink performance interior to the node is excellent.

Grid/benchmarks/Benchmark\_comms\_host\_device

Grid/MPI\_benchmark/halo\_mpi.c. (standalone )

## Use of coding agents in HPC development

### Parallel FFT performance tuning in Grid - happy to compare notes with, say, SKA or others

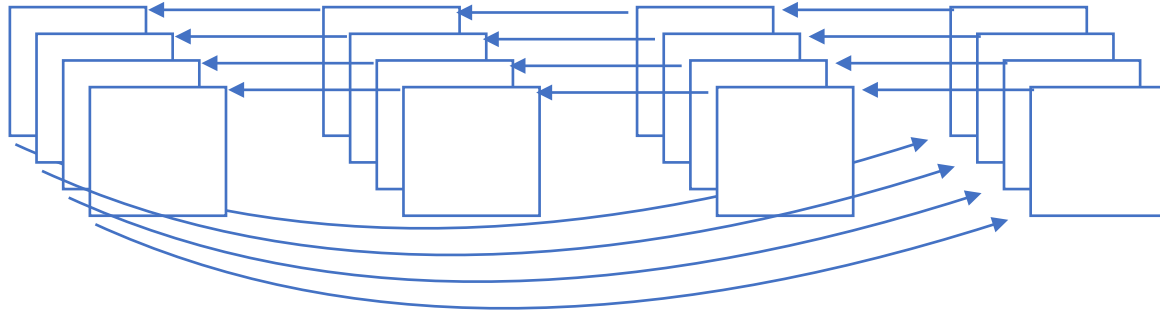
- Last year I offloaded to GPU, but didn't fully optimize communications
  - Also PETSc colleagues found FFTW is broken for  $D > 3$  !
  - I presented the FFT plan at your January DiRAC workshop; here it is finished.
    - Claude Fable took this plan and finished it with me in 1 day of work

### Codesign analysis:

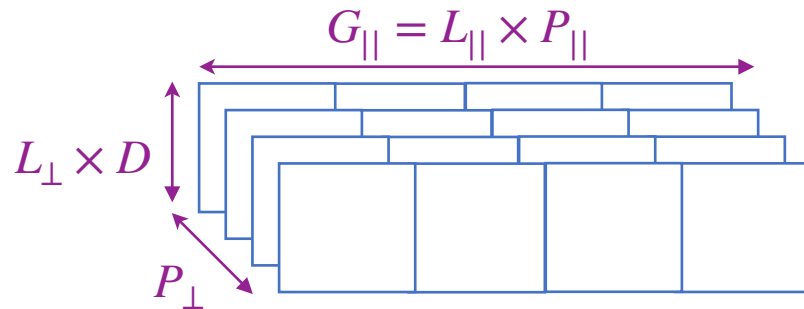
- FFT each node must send / receive entire local volume (N) of data (and no more)
  - For modest N,  $\log N$  is not a large number, local flops not huge and memory bandwidth for  $O(N)$  is key
  - Ratio of GPU HBM bandwidth to network is 80:1 on Frontier
  - Parallel FFT is necessarily comms limited.
    - Dslash: communicates surface, calculates volume.
    - FFT communicates volume, calculates volume ( $\times \log L$ )
  - Performs custom cartesian torus ring all-to-all at wire speed: cannot further speed up!
    - Some outcomes dictated by network procurement, not by optimization

## Perform D-dimensional FFT as **D sequential 1D FFTs**

- $P_{||} \times P_{\perp}$  processors,  $L_{||} \times L_{\perp}$  local volume and  $G_{||} \times G_{\perp}$  global volume. Arbitrary dof's D per site
  - “Pencil” approach to MPI parallelism: barrel shift and collect data locally



- $P_{\perp} \times L_{\perp} \times D$  local FFT's length  $G_{||}$  performed concurrently on GPUs; MPI and data always GPU resident
  - Always transpose data to make  $G_{||}$  contiguous (copy overhead)

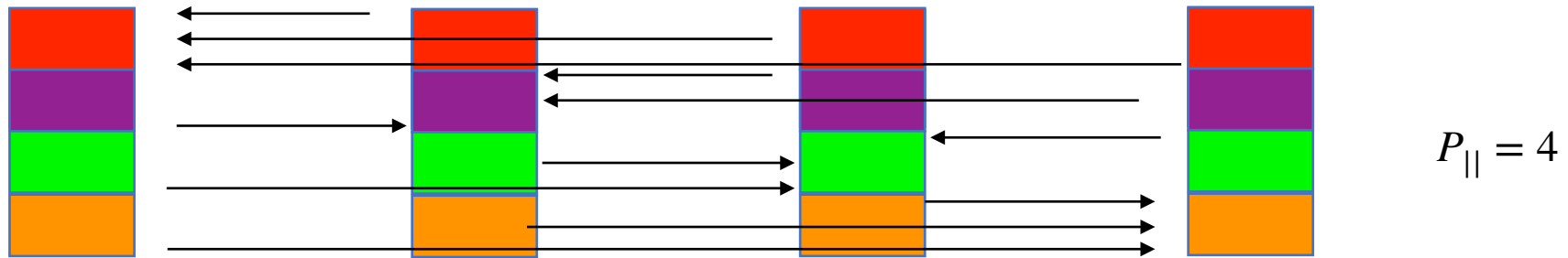


This is still not fully efficient as  $P_{||}$  fold redundant copies  
No comms penalty for  $P_{||} \leq 2$  as avoid redistributing results

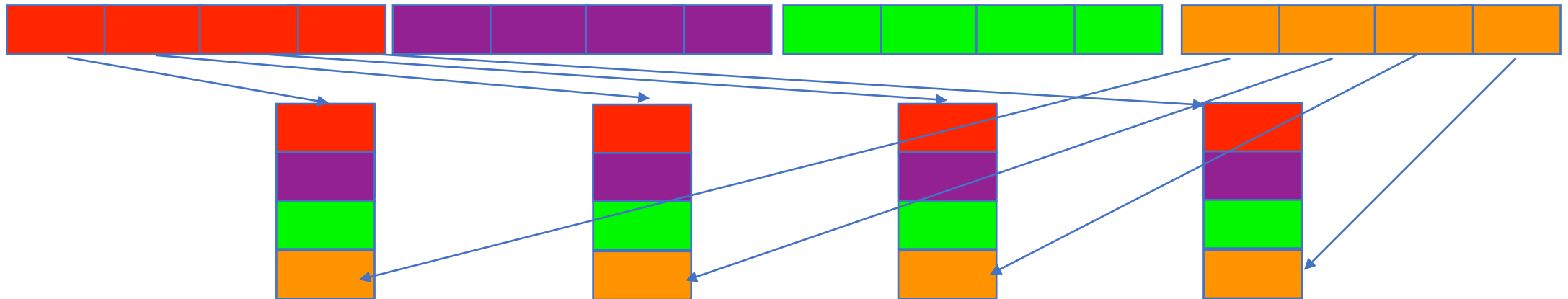
- 2x2x2x4 to 8x8x8x12 GPU's common ; so “good enough” for QCD

Could replace barrel shift with work division

- Each node sends  $\frac{1}{P_{||}}$  of it's volume to  $P_{||} - 1$  "neighbors" (AllToAll)



- Each node performs  $\frac{1}{P_{||}}$  of the FFT's - this will gain nothing as GPU subdominant
- Redistribute the results



- Communications cost improved from:  $P_{||} - 1$  to  $2 \frac{P_{||} - 1}{P_{||}}$  : only gain if  $P_{||} \geq 3$  !

## Frontier: 8 MPI rank AMD test system

4 x MI250 vs 64 cores gave **30x speed up vs. old code**

**48<sup>3</sup> x 96 FFT of 144 batch size ComplexD.**

CPU resident old code: 35.2s.

CPU resident new code: 34.8s

GPU resident new code: 1.19s

- Local FFT is sub-dominant and gives 1-2TF/s per half GPU in double precision
  - **Further 5x comms gain since January using Claude (depends on node count)!**
- **In a position to talk to SKA or others about performance**
  - **Claude read my Jan “Building Novel Methods” slides and then carried out phase 2**
  - **Implemented a suite of “ring” cartesian collectives (Alltoall, Allreduce, Allgather)**

- **Coding agents: enormous labour saver**

- Substantial investment in 'taming the tool'

- Establish 'memories' : e.g.

- do not guess, base on evidence
- use Grid facilities wherever possible, do not reinvent wheel or duplicate existing functionality
- Do **not** assume memory is accessible, follow rules for GPU memory vs. CPU memory.
- Do not perform git commits, always review diffs
- KISS, pointer / resource management rules etc...

- Push back:

- Don't be a push-over.
- Do not defer.
- Tell it what you want.
- Makes it make memories and change behavior longer term.

- Write a text document planning HOW you want code changed, phases and intermediate tests

- Tell claude the sequence you would follow if you were developing a feature
- Give an example of a related piece of code or style you want to follow

## Memories:

- Outlive “context”
- Outlive a new session in same directory
- Specific to project directory
  
- Associative memory index: recalls a memory when conversation/context suggests it is relevant!

Claude made these while working with me

### User (1)

- User background — IBM BG/Q chip designer (prefetch engine, memory interface); author of Grid; expert across the full HPC stack.

### Feedback — how to work (18)

- **Epistemic discipline** — flag speculation vs deduction; "I think" when unproven.
- **Test frequently** — validate each change before stacking; read logs, cause is often structural/older.
- **Look at the data** — observe actual state before theorizing
- Check basics are right first — verify sanity/preconditions before explaining phenomena.
- No git interaction — read-only git only; PB commits/pushes.
- getenv banned — no env knobs.
- Diff after every edit — read git diff, count hunks vs intent before building.
- **Don't re-contradict corrections** — search own context before re-explaining a mechanism.
- **Capture the concepts in comments** — register the conceptual/physics spine, not just debug facts.
- Comments: state not history — **comments describe current state, never history/bug-dates.**
- Respect optimised paths — **never simplify away a GPU optimisation** to fix a fallback; fix additively.
- **KISS raw-pointer design** — raw owning ptr (null→lazy→delete), assert invariants.
- **Stay on Grid primitives** — prefer GridBLAS/existing over naked roc\*/cu\* calls.
- Debug changes & audit scope — **debug scaffolding isn't submittable**; audit by shared resource.
- **Golden rule: minimal external dependence — least external functionality consistent with goals.**

### Project — ongoing work / results (19)

- Multigrid design notes — full design/tuning record (HDCG, PVdagM, coarse-op GPU, Lüscher deflation, Frontier data).
- Air-gapped workflow — Claude on laptop, Grid runs on Frontier; hand precise observation requests.
- Supercoarse + dense bottom — THE win: one-step block to dense floor held outer=34.
- 2D inverse comms-bound → RESOLVED — partial ring participation; LEAF\_SPAN=9 big leaves.
- FFT plan cost is huge on HIP — Frontier plan create/destroy ~22ms; PlannedFFT = 5x.
- Analyticity: PVdagM vs M smoother — GCR reduces PVdagM IR not M IR; analyticity is WHY MG on PVdagM.

### Reference — external facts / pointers (16)

- Registration/NUMA minefield — 10+ yrs of MPI+GPU registration/NUMA bugs; verification culture.
- Frontier GPU wrapper — never strip select\_gpu wrapper; forces SDMA peer copies.
- mRHS shared-coefficient rationale — shared-coeff GCR beats block for DWF (dense spectrum).
- Unkrylov / analyticity — poly smoothers: order=wiggles, range=amplitude; shift places boundary.
- Frontier NO\_TRANSLATION ledger — full investigation; no fabric knob changes it.
- FFT/QCD bandwidth & co-design — Dslash surface vs FFT volume comms; co-design the network.

## Two months work with Claude!

- **Two bugs in libfabric** (associated with MPI on Frontier@ORNL)
  - <https://github.com/ofiwg/libfabric/issues/12775>
  - <https://github.com/ofiwg/libfabric/issues/12773>
  - Claude wrote simple reproducer and located line of code in libfabric on second of these
    - I instructed Claude on the correct debug strategy and ran the jobs (fence posting)
- **10x improvement over MPI-2 I/O on collective buffered cartesian arrays MPI\_File\_read\_all**
  - <https://github.com/pmodels/mpich/issues/7935>
  - 20-40GB/s → 400GB/s
- **10x improvement in distributed dense matrix inversion over Exascale SLATE library on Frontier**
- **3 x improvement over MPI collectives for Alltoall, allreduce, all gather**
  - Dimension order Cartesian ring primitives in Grid hit wire-speed on Frontier
- **New Lexicographic Lattice options: Internal x First Grid dim guarantees contiguous matrix for BLAS**

**All supporting my multigrid R&D programme.**

# Summary

- Grid Software is **portable and performant on a broad range of architectures**
  - Software design is important
  - After a lot of work, we run efficiently on many different high end computers around the world
- **Vendor API's** can accelerate local computation and should be used where possible
  - **Batch GEMM** accelerated key parts of multigrid and deflation in LQCD
  - **Vendor local FFT's** are exceptionally fast
  - With **coding agents**, developing faster versions of “library functionality” is feasible and affordable!
    - **Matrix inversion; parallel FFT ; Collectives: reduction, alltoall, all gather**
- **Data motion between nodes remains problematic**
  - Compute speed is growing faster than interconnect speed
  - Have had to **rely less and less on advanced MPI features**
    - Use low level communications between GPUs in same node
    - Options to avoid combining MPI with device memory
      - Bugs and performance issues found on various systems; some rare and non-deterministic
- **The level of optimization required to accompany algorithm development to make practical is genuinely *distressing***
- ***But this is now substantially easier with coding agents!***